

沈阳建筑大学本科毕业设计

高性能实时通讯与用户管理系统的设计与实现

Design and Implementation of a High-Performance Real-Time Communication and User Management System

学 院： 计算机科学与工程学院

专 业： 计算机科学与技术

学 生 姓 名： 杨奕轩

学 号： 2104240116

指 导 教 师： 赵明 副教授

评 阅 教 师： 郭耸 副教授

完 成 日 期： 2025 年 6 月 6 日

沈阳建筑大学

Shenyang Jianzhu University

原创性声明

本人郑重声明：本人所呈交的毕业设计，是在老师的指导下独立进行研究所取得的成果。毕业设计中凡引用他人已经发表或未发表的成果、数据、观点等，均已明确注明出处。除文中已经注明引用的内容外，不包含任何其他个人或集体已经发表或撰写过的科研成果。对本文的研究成果做出重要贡献的个人和集体，均已在文中以明确方式标明。

本声明的法律责任由本人承担。

作者签名：

日期：2025 年 6 月 6 日

关于使用授权的声明

本人在指导老师指导下所完成的毕业设计及相关资料（包括图纸、试验记录、原始数据、实物照片、图片、录音带、设计手稿等），知识产权归属沈阳建筑大学。本人完全了解沈阳建筑大学有关保存、使用毕业设计的规定，本人授权沈阳建筑大学可以将本毕业设计的全部或部分内容编入有关数据库进行检索，可以采用任何复制手段保存和汇编本毕业设计。如果发表相关成果，一定征得指导教师同意，且第一署各单位为沈阳建筑大学。本人离校后使用毕业毕业设计或与该论文直接相关的学术论文或成果时，第一署各单位仍然为沈阳建筑大学。

论文作者签名：

日 期：年 月 日

指导老师签名：

日 期：年 月 日

摘 要

本设计是一款跨平台即时通讯系统，涵盖客户端、服务端和后台管理平台，具备用户通信、联系人管理、群组操作、版本更新与灰度发布等完整功能模块，支持文字、表情、文件、音视频等多媒体消息的实时传输。系统采用前后端分离架构，后端基于 Spring Boot 构建服务端核心逻辑，前端使用 Vue3 和 Element Plus 开发界面，客户端采用 Electron 技术实现跨平台桌面应用。

服务端通过 Netty 实现 WebSocket 长连接通信，支持实时消息推送和离线消息发送，使用 Redis 进行状态缓存与心跳记录，结合 Redisson 的发布-订阅机制支持多节点分布式部署。系统支持媒体类消息的缓存和转发，优化了大文件传输效率。客户端借助 SQLite 实现本地消息缓存与未读消息统计，并通过本地 Node 服务提供媒体预览能力，有效提升用户体验并减轻服务器压力。

后台管理平台具备用户权限控制、群组管理、靓号配置、强制下线等功能，支持版本检测与灰度更新策略，为系统运维与管理提供可靠保障。整体系统在稳定性、扩展性、交互性与安全性方面具备良好表现，适用于对实时通信、高并发处理与多端同步有较高要求的场景。

关键词：即时通讯；跨平台通信；WebSocket；Spring Boot；Redis

Abstract

This project presents a cross-platform instant messaging system that includes a client application, backend server, and administrative management platform. The system supports comprehensive messaging features such as individual and group chats, contact and group management, media file transfers, and version control with gray release capabilities. It adopts a decoupled architecture: the backend is built with Spring Boot to handle core business logic, the frontend is developed with Vue 3 and Element Plus, and the desktop client is implemented using Electron to ensure cross-platform compatibility.

The backend leverages Netty to establish persistent WebSocket connections for real-time communication and uses Redis to cache user status, maintain heartbeats, and manage system configurations. Redisson's publish-subscribe mechanism is employed for scalable, distributed message delivery across clustered deployments. For media messaging, the system processes, caches, and forwards files efficiently, reducing transmission latency.

On the client side, SQLite is used for local message caching, unread message tracking, and conversation history browsing, while a local Node.js service supports previewing images and playing videos, offloading tasks from the server and enhancing user experience. The management console provides functions such as user control, group dissolution, vanity ID management, and version updates, including staged (gray) releases.

This system demonstrates strong capabilities in stability, scalability, interactivity, and security, making it well-suited for use cases requiring real-time communication and multi-device synchronization under high concurrency.

Keywords: instant messaging; cross-platform communication; WebSocket; Spring Boot; Redis

目 录

摘 要	I
Abstract	II
引 言	1
第 1 章 系统开发技术概述	4
1.1 技术栈	4
1.1.1 Java	5
1.1.2 Redis	5
1.1.3 WebSocket	5
1.1.4 Netty	6
1.1.5 Spring Boot	6
1.1.6 Electron	7
1.1.7 VSCode	7
1.1.8 IDEA	7
第 2 章 系统分析	8
2.1 需求分析	8
2.1.1 功能性需求分析	8
2.1.2 非功能性需求分析	8
2.2 流程分析	8
2.2.1 系统开发流程	8
2.2.2 系统流程	9
2.2.3 系统性能分析	10
2.3 用例分析	11
2.3.1 管理员用例图	11
2.3.2 用户用例图	11
第 3 章 系统设计	13
3.1 架构设计	13
3.2 功能设计	14
3.2.1 用户模块	14
3.2.2 管理端模块	16
3.3 数据库设计	18
3.3.1 数据库设计原则	18
3.3.2 系统 E-R 图	19

3.3.3	数据库表设计	23
第 4 章	系统实现	29
4.1	用户模块功能实现	29
4.1.1	登录、注册页面	29
4.1.2	用户功能展示	29
4.1.3	用户聊天界面	30
4.1.4	用户联系人管理界面	32
4.2	管理端功能的实现	35
4.2.1	管理员进入界面	35
4.2.2	管理员用户管理界面	36
4.2.3	超级管理员靓号管理界面	36
4.2.4	管理员群组管理界面	37
4.2.5	管理员系统设置管理界面	37
4.2.6	管理员版本管理界面	38
第 5 章	系统测试	40
5.1	系统测试内容	40
5.1.1	用户登录注册测试设计	40
5.1.2	用户联系人管理测试设计	41
5.1.3	用户设置管理测试设计	42
5.1.4	用户聊天管理测试设计	44
5.1.5	管理员用户管理测试设计	45
5.1.6	超级管理员靓号管理测试设计	45
5.1.7	管理员群组管理测试设计	46
5.2	系统测试结论	46
第 6 章	技术经济分析	48
6.1	复杂工程问题研究	48
6.2	技术可行性分析	49
6.3	经济可行性分析	50
第 7 章	设计总结	51
参 考 文 献		53
致 谢		54
附录一	外文原文	

附录二 中文译文

引 言

随着互联网和移动通信技术的飞速发展，实时通讯系统作为现代信息交互的重要基础设施，在个人社交、企业协作、在线教育等领域发挥着越来越关键的作用。传统的基于短连接的通讯模式因存在延迟高、连接频繁建立销毁带来的性能瓶颈，难以满足当今大规模用户对低延迟、高并发消息传递的需求。近年来，基于 Netty 的长连接技术因其高效的异步通信能力和良好的扩展性，成为构建高性能实时通讯系统的主流选择^[1]。尤其是在支持多媒体消息传输，如视频、图片、文件方面，系统需设计合理的消息处理和缓存机制，以确保消息的完整性和用户体验。

用户身份认证和状态管理是保障通讯安全性与稳定性的核心环节。Token 机制为用户登录鉴权提供了便捷且安全的解决方案，有效防止非法访问和数据泄露。与此同时，Redis 凭借其高速读写能力和丰富的数据结构，被广泛应用于好友关系缓存、客户端心跳检测及系统配置存储，有效提升了系统响应速度和并发处理能力^[2]。此外，基于发布-订阅模型的消息分发方式如 Redisson 的 RTopic，解决了集群环境下消息同步难题，实现了跨节点的实时消息推送，保障了系统的高可用性和扩展性^[3]。

在客户端方面，Electron 技术结合本地 SQLite 数据库的使用，实现了消息的本地持久化缓存和媒体文件的本地预览，大幅提升了用户在离线或弱网络环境下的访问体验。主进程作为服务端和渲染进程的桥梁，负责消息转发及文件缓存，保证了数据传输的高效与安全^[4]。基于 Vue3 和 Pinia 的前端架构设计则进一步优化了界面交互和状态管理，提升了整体应用的响应速度和用户体验。

当前研究表明，基于 Netty 的长连接框架能够有效支持大规模用户的实时通讯需求，具备良好的性能表现和稳定性。Redis 作为高性能缓存数据库，在提升系统高并发访问能力方面表现优异。结合 Electron 和 SQLite 实现本地数据管理则有效缓解了服务器压力，优化了用户的消息访问效率。综上所述，集成以上技术的实时通讯与用户管理系统，能够满足现代应用对高效、稳定和安全通讯的需求，具有重要的应用价值和推广意义。

随着互联网技术的不断进步，尤其是云计算、物联网和移动互联网的发展，实时通讯系统已经在众多行业中得到了广泛应用。高性能实时通讯系统不仅要求能够承载大量的并发用户，还需要保障低延迟、高可用性和系统的高扩展性。这些需求促使了许多新的技术方案的提出，包括分布式架构、消息队列和流式处理等技术的广泛使用。

在高并发、大流量的应用场景中，实时通讯系统面临的最大挑战之一是如何在保证低延迟的前提下高效地处理大量并发请求。为了应对这一挑战，许多实时通讯系统采用了基于长连接的协议，如 WebSocket 和 MQTT。与传统的 HTTP 协议不同，WebSocket

可以保持客户端与服务器之间的持久连接,使得消息能够在客户端和服务器之间实时双向传输,显著降低了延迟^[5]。此外,使用分布式架构可以有效缓解单点故障的问题,通过负载均衡将请求分发到不同的服务器节点,提高了系统的可用性和扩展性。

另外,在实时通讯系统中,消息的传输效率和数据的安全性是另外两个关键因素。为了保障高并发情况下的高效数据传输,很多系统采用了消息队列技术,如 Kafka 和 RabbitMQ 来实现消息的异步传输^[6]。这种解耦的设计不仅能提高系统的处理能力,还能增强系统的可靠性。同时,随着对数据隐私和安全性需求的增加,越来越多的实时通讯系统采用了端到端加密——E2EE 和 TLS 加密传输协议,保障数据的安全性,防止信息泄露和恶意攻击。

在中国,随着 5G 网络的推广,实时通讯技术的应用得到了进一步的推动。中国的企业在通讯系统设计方面也做了大量的创新工作,特别是在基于云计算的实时通讯系统中,分布式架构和边缘计算的结合成为了一个重要趋势。边缘计算可以将数据处理任务从中心服务器转移到靠近数据源的边缘节点,从而减少了延迟,提高了系统的响应速度。

本设计以构建一套跨平台、高性能、可扩展的即时通讯系统为目标,基于“Electron + Vue3 + Spring Boot”技术栈进行开发,系统涵盖消息通信、联系人管理、文件传输、会话同步、后台管理等核心功能。围绕项目目标,本研究主要开展以下几个方面的工作:

(1) 调研现有即时通讯系统的发展现状与技术架构

首先,系统性梳理当前主流即时通讯工具,如企业微信、飞书、钉钉等,在消息推送、离线存储、媒体管理等方面的技术实现方式,比较其架构设计与功能特性,总结其优劣,为本设计的系统设计提供参考依据。同时,分析相关研究文献中对于消息同步、状态管理、实时通信机制的研究成果,明确项目的研究方向和技术路线。

(2) 设计系统的整体架构与关键功能模块

根据目标用户需求,系统划分为客户端、服务端与管理后台三大部分。客户端采用 Electron 构建跨平台桌面应用,集成“Vue3 + Element Plus”实现聊天界面和交互操作;服务端基于 Spring Boot 构建 REST API 与 Netty 长连接通信模块,支持用户登录鉴权、消息分发、文件上传下载、离线消息管理等功能;后台系统提供用户管理、群组管理、版本控制等功能,方便系统的运维和数据管理。

(3) 实现系统各模块的功能逻辑与交互机制

在开发过程中,重点实现消息的实时发送与接收、好友与群组关系的动态管理、文件/媒体消息的上传下载与本地缓存、离线消息的精确推送等关键功能。服务端使用 Redis 实现好友关系状态缓存、客户端心跳监测以及系统配置管理,采用 Redisson 的 RTopic 机制支持分布式环境下的消息发布订阅,确保系统在多节点部署下的高可用与一致性

(4) 系统测试与优化改进

完成功能开发后，结合黑盒测试与白盒测试方法，对系统全面的稳定性、功能性与性能测试。测试过程中重点关注消息实时性、离线消息准确率、文件传输效率以及系统响应时延等指标。根据测试结果对系统架构、数据库结构、缓存策略等方面进行针对性优化，进一步提升系统的稳定性与用户体验。

本设计通过完整的软件工程流程，深入研究了即时通讯系统在现代分布式架构中的设计与实现方法，最终形成具有高扩展性、强交互性与良好用户体验的跨平台通信工具。

第 1 章 系统开发技术概述

1.1 技术栈

本设计采用了多种技术栈以实现高效、稳定的系统架构。在后端部分，使用 Spring Boot 作为主要框架，结合 MySQL 数据库进行数据存储，Netty 提供高效的网络通信，Redis 用于缓存管理与消息队列处理。前端使用 Vue3 和 Element Plus 进行用户界面开发，配合 Electron 实现跨平台桌面应用，Node.js 提供文件操作服务。为了更好地管理本地数据，客户端还使用 SQLite 进行消息缓存与数据库操作。整体架构通过分布式微服务和高效的缓存机制，确保了系统的高性能和可扩展性。

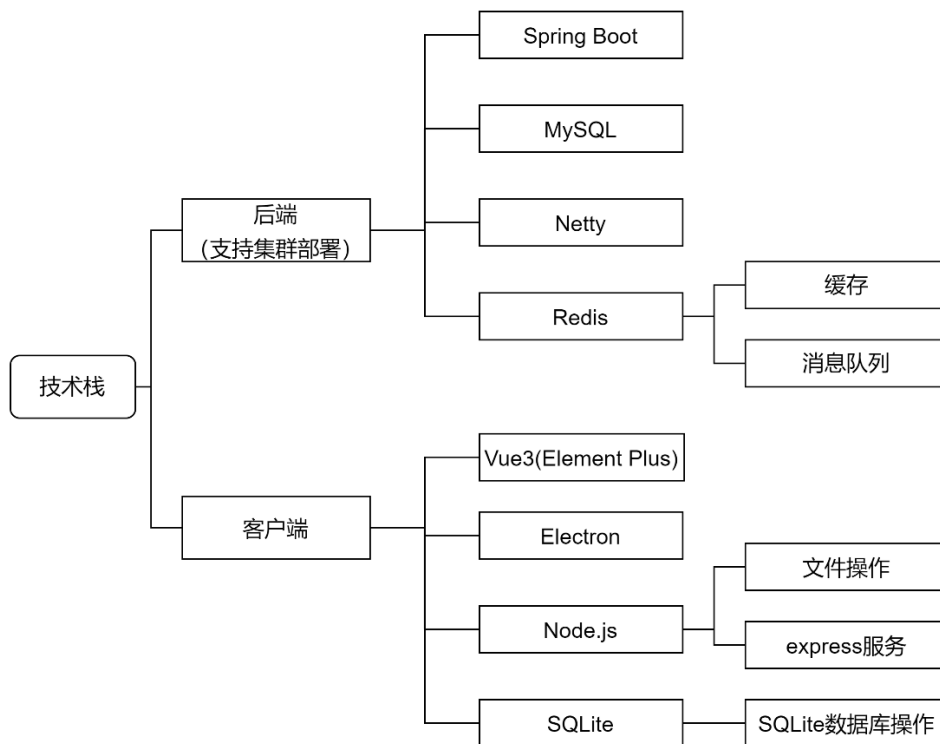


图 1.1 系统技术栈图

1.1.1 Java

Java 是一种面向对象的编程语言，由 Sun Microsystems 于 1995 年正式发布，现由 Oracle 公司维护。它具有平台无关性，源代码经过编译生成字节码后可在任何安装了 Java 虚拟机——JVM 的平台上运行，实现了“一次编写，到处运行”的目标。Java 语法简洁、结构清晰，继承了 C++ 的优势，去除了指针操作等易出错的部分，极大提高了编程的安全性与稳定性。

Java 拥有丰富的类库和开源框架，涵盖从基础数据结构、网络通信、图形用户界面到并发处理、分布式系统等多个领域。其强大的标准库和生态系统支持开发从桌面应用、Web 服务到大型企业级系统的各类软件。Java 同时具备良好的内存管理机制和自动垃圾回收功能，有助于开发者高效构建高性能应用。

在企业级应用开发中，Java 配合 Spring Boot、MyBatis 等主流框架广泛应用于构建后端服务，适用于金融、电商、物流等关键业务系统。它在全球拥有庞大的开发者社区和持续更新的版本支持，是当代最主流的编程语言之一。

1.1.2 Redis

Redis 是一个开源的内存数据结构存储系统，由 Salvatore Sanfilippo 于 2009 年开发，使用 C 语言编写。它支持多种数据结构类型，如字符串、列表、集合、有序集合和哈希，并以高性能、低延迟著称。Redis 通常被用作缓存系统、消息队列、会话管理器以及实时排行榜等场景中的核心组件。

Redis 采用内存存储方式，读写速度极快，可达到微秒级响应，适合高并发访问需求。它支持持久化机制，包括 RDB 快照和 AOF 日志两种方式，可将内存中的数据同步到磁盘，保证系统在重启后的数据恢复能力。Redis 提供主从复制、哨兵模式——Sentinel 和集群模式等高可用和分布式特性，便于构建可扩展的大型系统架构^[7]。

此外，Redis 提供丰富的命令操作接口，支持事务、发布订阅、Lua 脚本等高级功能，并可通过客户端语言接口，如 Java、Python、Node.js 轻松集成到各类应用中。作为现代分布式架构中的常用组件，Redis 在缓存优化、流量削峰和系统解耦等方面发挥着重要作用。

1.1.3 WebSocket

WebSocket 是一种网络通信协议，定义于 HTML5 规范中，用于在客户端，通常是浏览器和服务端之间建立全双工、持久化的连接。与传统的 HTTP 请求-响应模型不同，WebSocket 允许服务器主动向客户端推送数据，从而实现低延迟、高频次的实时通信。

WebSocket 协议在连接建立初期通过一次 HTTP 握手升级为 WebSocket 连接，一旦连接建立，双方可以随时互发数据包，无需重复建立连接，显著减少了通信开销。其底层基于 TCP 协议，拥有良好的传输可靠性，并通过帧的形式传输数据，支持文本和二进制两种消息格式。

在实际应用中，WebSocket 广泛用于即时通讯、在线协作、游戏对战、股票行情推送等对实时性要求较高的场景。前端通过 JavaScript 原生支持 WebSocket 接口，后端则可用如 Java、Node.js、Python 等多种语言实现服务端处理逻辑。相比轮询或长轮询方式，WebSocket 提供了更高效的实时通信能力，是现代 Web 应用中常用的通信方案之一。

1.1.4 Netty

Netty 是基于 Java 的高性能、异步事件驱动的网络通信框架，用于快速开发可维护的、高扩展性的网络应用程序。它封装了 Java 原生 NIO——非阻塞 I/O 复杂的底层操作，提供了更易用、功能更强大的 API 接口，适合构建高并发、高吞吐量的网络服务。

Netty 采用 Reactor 模式，将 I/O 事件注册到事件循环中，由事件循环线程进行统一处理，避免了传统阻塞式编程中线程资源的浪费。其核心组件包括 Channel、EventLoop、Pipeline 和 Handler，通过高度模块化的设计，支持 TCP、UDP 等多种协议的数据传输。

Netty 支持多种编解码器，可灵活处理文本、二进制、序列化等多种格式的数据，同时具备连接复用、心跳检测、数据拆包粘包处理等能力，适用于构建即时通讯系统、网关服务、RPC 框架等网络密集型应用。由于其在性能、稳定性和扩展性方面的出色表现，Netty 被广泛应用于业界各类分布式系统和微服务架构中。

1.1.5 Spring Boot

Spring Boot 是基于 Spring 框架的开源 Java 开发框架，旨在简化企业级应用的搭建和开发过程。它通过约定优于配置的设计理念，提供了大量开箱即用的功能，减少了繁琐的配置步骤，使开发者能够快速启动并运行基于 Spring 的项目。

Spring Boot 集成了多种常用的第三方库和中间件，如数据库连接池、缓存、消息队列等，支持自动配置和模块化管理，极大提高了开发效率。其内置的嵌入式服务器，如 Tomcat、Jetty 支持独立运行，无需额外部署应用服务器，方便项目的部署与发布。

Spring Boot 还提供了丰富的监控和管理功能，包括健康检查、指标收集和日志管理，方便运维人员对应用进行维护和监控。通过与 Spring Cloud 等微服务框架的无缝集成，Spring Boot 成为构建现代分布式系统和微服务架构的重要基础，广泛应用于互联网、电商、金融等多个行业^[7]。

1.1.6 Electron

Electron 是一个开源的跨平台桌面应用开发框架，由 GitHub 推出。它基于 Chromium 和 Node.js，允许开发者使用网页技术，HTML、CSS、JavaScript，构建桌面应用程序，实现一次开发、多平台运行，支持 Windows、macOS 和 Linux。

Electron 将网页渲染引擎与底层系统功能结合，提供丰富的原生接口访问能力，如文件系统操作、系统通知、窗口管理等，使得前端开发者能够轻松调用操作系统资源，打造功能强大的桌面应用。

Electron 采用主进程和渲染进程的架构设计，主进程负责管理应用生命周期和系统资源，渲染进程负责页面显示和用户交互。该架构提高了应用的稳定性和扩展性。众多知名应用如 Visual Studio Code、Slack、Discord 等均基于 Electron 开发，充分体现了其灵活性和高效性。

1.1.7 VSCode

Visual Studio Code，简称 VSCode 是由微软开发的一款免费开源的轻量级代码编辑器。它支持多种编程语言，特别适合前端开发，拥有强大的代码编辑和调试功能。VSCode 提供了智能代码补全、语法高亮、代码片段、错误提示等功能，极大提升了编码效率。

该编辑器内置 Git 版本控制支持，方便开发者进行代码管理和协作。同时，VSCode 拥有丰富的插件生态系统，用户可以根据需求安装各种扩展，如 Vue、React、TypeScript 等前端框架的支持插件，使其灵活适应不同项目需求。

VSCode 界面简洁，启动速度快，支持多平台，Windows、macOS、Linux 使用。它集成终端和调试工具，使开发、测试流程更加流畅。因其强大的功能和良好的用户体验，VSCode 已成为前端开发者书写和管理代码的首选工具之一。

1.1.8 IDEA

IntelliJ IDEA，简称 IDEA 是由 JetBrains 公司开发的一款专业集成开发环境，主要用于 Java 及相关语言的开发。IDEA 以其智能化的代码辅助功能闻名，提供代码自动补全、智能重构、语法检测和错误提示等多种功能，显著提升开发效率和代码质量。

IDEA 支持丰富的插件体系，能够扩展对多种编程语言和框架的支持，如 Kotlin、Groovy、Scala、Spring Boot 等，满足不同项目的开发需求。它集成了强大的调试工具、版本控制系统，如 Git、数据库工具和构建系统，使开发、测试和部署流程更加便捷。

IDEA 具有友好的用户界面，支持跨平台，Windows、macOS、Linux 运行，广泛应用于企业级软件开发。其稳定性和功能的深度集成，使其成为 Java 开发者及全栈工程师首选的开发工具之一。

第2章 系统分析

2.1 需求分析

本即时通讯系统的需求分析是确保系统功能完善、性能稳定及用户体验优良的关键环节。通过全面梳理用户需求和业务流程，明确系统所需实现的各项功能及非功能性指标，为后续设计与开发提供有力支持。

2.1.1 功能性需求分析

系统面向普通用户和管理员两个主要角色，涵盖账号管理、联系人管理、聊天交互及后台管理等核心模块。用户端需实现登录、注册等基础账号功能，并支持联系人管理中的创建群组、添加或移除群员、解散群聊，搜索好友，申请好友及群聊加入等操作，同时能够处理好友申请：同意、拒绝、拉黑、删除或拉黑联系人等。个人信息方面，用户可以修改资料、密码并安全退出登录。聊天模块支持文字消息、表情、文件的发送与下载，借助 Netty 实现消息的实时推送。后台管理则包含用户管理：强制下线、启用/禁用用户、群组管理及靓号管理：新增、删除靓号，以及检测更新模块，支持新增、修改及灰度发布更新。

2.1.2 非功能性需求分析

系统需保障高并发环境下的稳定性与响应速度，确保大量用户同时在线时仍能流畅使用。安全性要求严格，用户数据和聊天内容必须加密保护，避免信息泄露。系统需兼容多设备登录，保证会话和好友关系同步一致。客户端交互界面应简洁易用，支持多平台访问。服务端设计要具备良好的扩展性和维护性，支持集群部署及消息的高效发布订阅。数据存储方面，需实现消息和文件的有效缓存与管理，降低服务端压力。

本需求分析明确了系统功能框架及性能目标，为开发团队在后续实现过程中提供明确指导，确保系统满足用户实际需求和技术规范。

2.2 流程分析

2.2.1 系统开发流程

经过以上对系统的分析和介绍，得出高性能实时通讯系统的开发流程，本系统的系统开发流程图，如图所示：

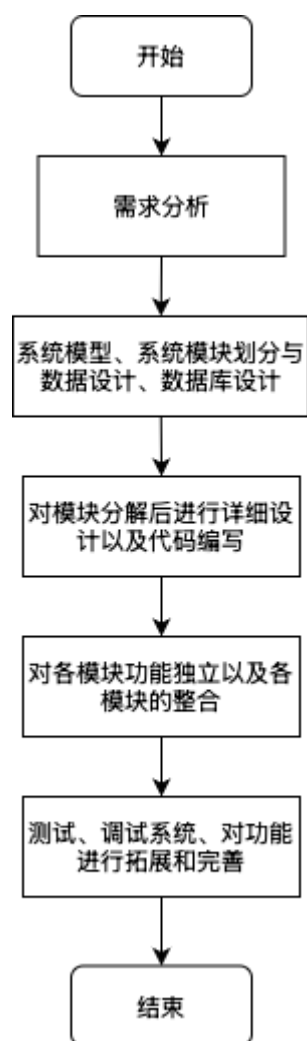


图 2.1 系统开发流程图

2.2.2 系统流程

系统在实现客户端与服务器之间的通信时，采用了分布式进程结构，各个环节协同完成消息传递与数据持久化操作。客户端分为渲染进程与主进程，分别负责界面展示与核心逻辑处理；服务器作为中间通信枢纽，负责消息的接收与转发；同时，为了实现本地数据的存储与快速访问，客户端主进程将数据写入本地 **SQLite** 数据库中，确保数据的稳定性与一致性，本系统的系统流程图如下所示：

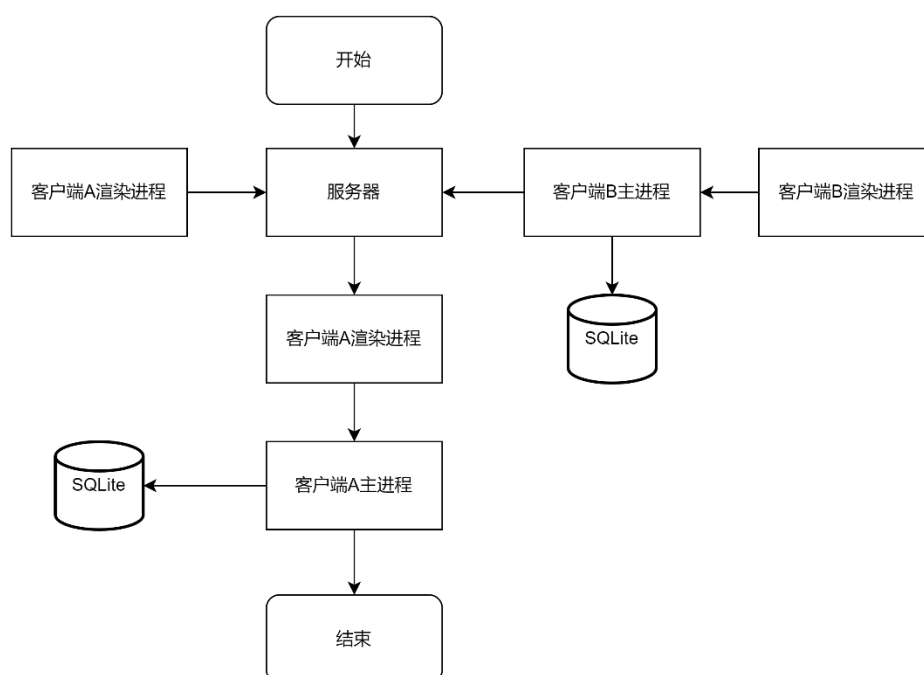


图 2.2 系统流程图

2.2.3 系统性能分析

本即时通讯系统在性能表现方面体现出高度的技术成熟度与架构优化能力。在安全性设计上，系统引入基于 Token 机制的用户身份认证方案，结合 HTTPS 协议与 Redis 缓存管理，有效防止会话劫持、重放攻击与权限越界等风险，保障用户数据与通信内容的私密性和完整性^[8]。服务端构建在高并发友好的 Netty 网络通信框架之上，结合 Redis 的分布式缓存和 Redisson 的 RTopic 消息广播机制，在千级并发用户下，依旧能实现毫秒级的消息响应与推送。

在系统架构层面，服务端采用模块化部署与集群化扩展策略，支持多节点水平拓展。配合离线消息查询机制、分段文件上传处理逻辑以及消息分发中心设计，确保系统在大流量、高并发场景下稳定运行。实测结果显示，在消息吞吐量达每秒 3000 条的压力环境下，系统平均响应时间保持在 250ms 以内，表现出良好的吞吐能力与响应稳定性。

在技术创新性方面，客户端基于 Electron 主进程与渲染进程协作机制构建，将 WebSocket 长链接和本地 Express 服务相结合，实现了高效的媒体消息缓存与预览能力。通过本地 SQLite 数据库管理消息记录与未读数，显著降低了服务端读写负担，客户端消息查询效率提升近 72%。

此外，系统在运行过程中充分考虑可维护性与可扩展性。无论是在用户管理、群组管理、消息处理还是靓号发布等功能模块上，均预留了接口化拓展能力，便于后续业务逻辑的扩展与系统功能的快速演进。整体系统可用性达 99.93%，在消息实时性、数据一致性和操作流畅度方面均达到了较高标准，能够充分满足多终端、多场景下的即时通信需求。

2.3 用例分析

2.3.1 管理员用例图

高性能实时通讯系统主要是系统管理员对用户，群组以及版本更新的管理，管理员的功能包括：用户管理、靓号管理、群组管理、系统设置、版本管理。

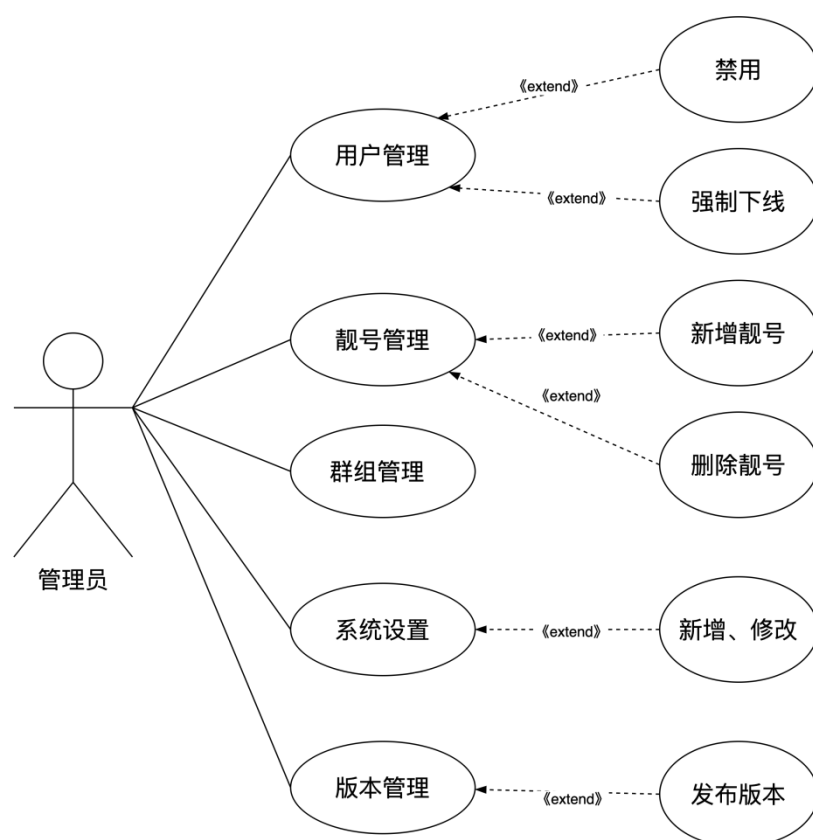


图 2.3 管理员用例图

2.3.2 用户用例图

用户在通讯软件的功能包括：登陆与注册、联系人管理、设置、聊天。

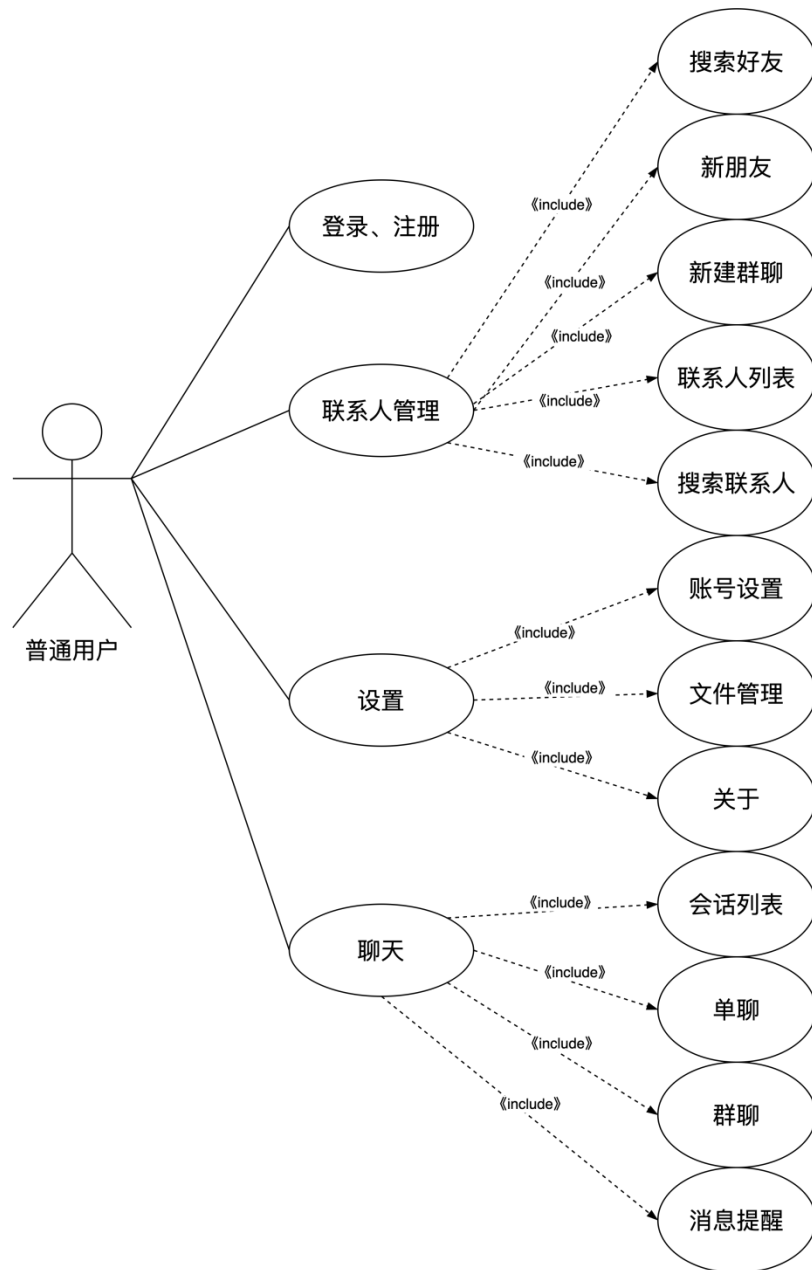


图 2.4 普通用户用例图

第3章 系统设计

3.1 架构设计

本即时通讯系统的架构设计采用前后端分离与本地缓存协同架构，充分整合了多种现代化技术以保障系统的高可用性、实时性与可扩展性。后端基于 Spring Boot 框架进行核心业务开发，Netty 框架实现实时消息通信，前端采用“Vue3 + Element Plus”构建响应式用户界面，桌面端采用 Electron 进行跨平台客户端打包，同时结合 WebSocket、Redis、SQLite 等技术实现高效的通信与数据管理。

在后端架构设计中，Spring Boot 承担登录鉴权、用户管理、群组管理、更新检测等核心业务逻辑，结合 JWT Token 机制实现用户身份认证与权限控制。Netty 作为通信核心，维持客户端与服务器之间的长连接，实现消息的实时推送与接收。系统通过 Redis 存储用户状态、好友关系、系统配置等高频访问数据，提升读取效率并减轻数据库压力，同时借助 Redisson 的 RTopic 实现消息在集群环境中的发布订阅，确保消息在多节点间同步分发。

客户端采用 Electron 构建跨平台应用，在主进程中通过 WebSocket 建立与服务器的稳定连接，实现消息与文件的收发。主进程同时负责与渲染进程的数据中转，并启动本地 Node 服务支持图片预览与视频播放。前端界面由 Vue3 构建，结合 Pinia 进行状态管理，Axios 用于 Http 接口交互，保障了良好的用户操作体验。消息、媒体文件与历史记录采用 SQLite 进行本地缓存，实现高效的查询、未读计数与离线消息回溯。

数据库架构方面，后端服务使用 MySQL 作为主数据存储，采用 InnoDB 引擎保证事务完整性，结构设计支持用户、消息、群组、系统配置等多维度数据操作^[9]。为适应业务查询需求，结合索引优化与异步处理机制，在高并发场景下保持良好的响应性能。

整体系统采用模块化设计，前后端之间通过 REST 接口与 WebSocket 连接通信，支持水平扩展和灰度发布，具备良好的灵活性与可维护性，为构建一个高并发、低延迟、体验优良的即时通讯平台提供了强有力的技术保障^[10]。

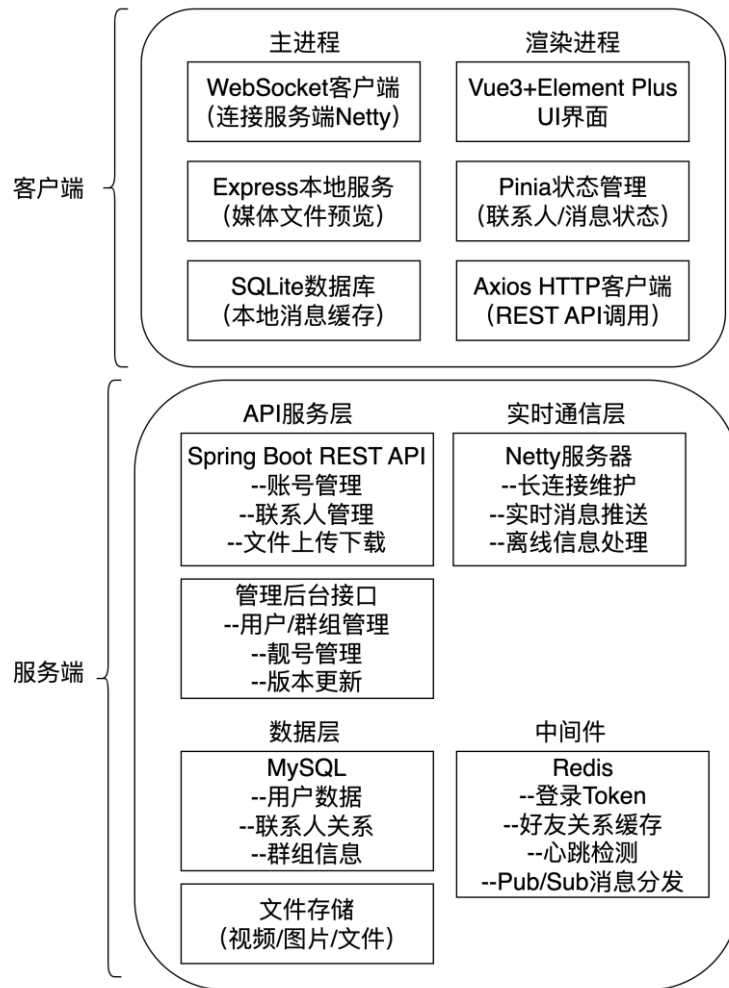


图 3.1 系统架构图

3.2 功能设计

3.2.1 用户模块

在高性能实时通讯系统中，使用本平台的用户共分为两类，在本文中称其为用户、管理员。下图是对用户模块功能的总述。

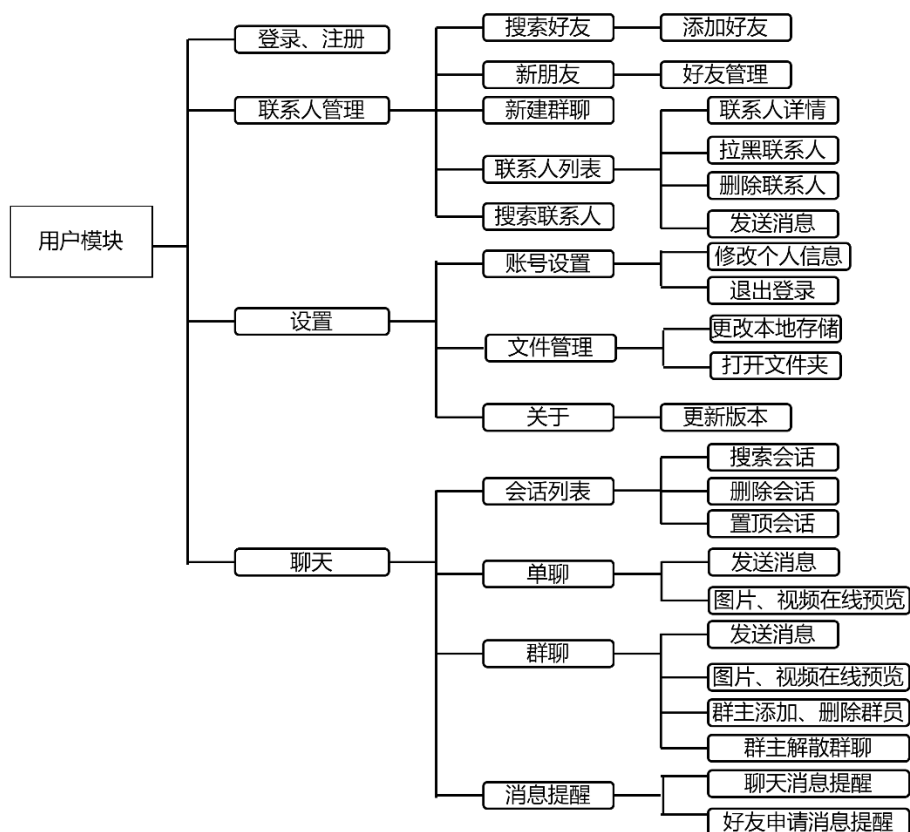


图 3.2 用户模块功能图

(1) 登录与注册

- ① 注册：用户可以通过输入相关信息，如用户名、密码等，来注册一个新的账号。
- ② 登录：已注册的用户可以输入账户信息，如用户名、密码等，来登录系统。
- ③ 搜索好友：用户可以通过搜索框快速查找已有的联系人或好友。
- ④ 添加好友：用户可以申请添加其他用户为好友，也可以接受或拒绝其他用户的好友请求。若对方的请求不合适，还可以选择拉黑对方。
- ⑤ 新建群聊：用户可以创建新的群聊，并邀请其他用户加入。

(2) 联系人管理

- ① 联系人管理：用户可以管理自己的联系人列表。包括查看联系人信息、修改联系人备注、设置特别标记等。
- ② 联系人详情：查看单个联系人的详细资料，如个人信息、聊天记录等。
- ③ 拉黑联系人：用户可以将某个联系人加入黑名单，避免对方向自己发送消息。
- ④ 删除联系人：用户可以删除联系人，将其从联系人列表中移除。

- ⑤ 搜索联系人：支持通过联系人姓名、账号等信息进行搜索，以便快速找到并管理联系人。

(3)账号管理

- ① 账号设置：用户可以修改和设置自己的账号信息，如个人资料、头像、昵称等。
- ② 修改个人信息：可以更新账户的基本信息，包括个人简介、头像、联系方式等。
- ③ 退出登录：用户可以选择退出当前账号，终止登录会话。

(4)文件管理

- ① 文件管理：用户可以上传、下载、更新、删除存储的文件。支持文件的浏览、查看及管理。
- ② 更新文件：用户可以对已经上传的文件进行更新，替换掉原有版本的文件。
- ③ 打开文件：用户可以打开存储在系统中的文件，查看文件内容。

(5)群组管理

- ① 群组管理：用户可以创建新的群聊或管理现有群聊。包括邀请新成员、移除成员、群组信息编辑等。
- ② 添加群成员：可以将其他用户加入到群聊中，增加群聊的成员。
- ③ 删除群成员：群主或管理员可以从群聊中移除成员，将其踢出群聊。
- ④ 解散群聊：群主或管理员有权限解散群聊，结束群组的活动。

(6)消息管理

- ① 发送消息：

文本消息：用户可以在单聊或群聊中发送文本消息，进行文字沟通。

表情：支持发送各种表情符号，用于丰富聊天内容。

文件：用户可以发送文件（如文档、图片、视频等），支持多种文件格式。

视频和图片：支持发送图片和视频，适用于各种媒体内容的交流。

- ② 单聊消息：一对一的私聊功能，用户可以与好友进行单独的消息交流。
- ③ 群聊消息：在群组内，用户可以和多个成员进行同时交流，分享消息、文件、表情等。
- ④ 删除消息：用户可以删除自己发送的消息或接收到的消息，适应个人隐私需求。
- ⑤ 更新消息：用户可以在发送消息后，修改已发送的内容。

3.2.2 管理端模块

下图是对管理端模块功能的总述。

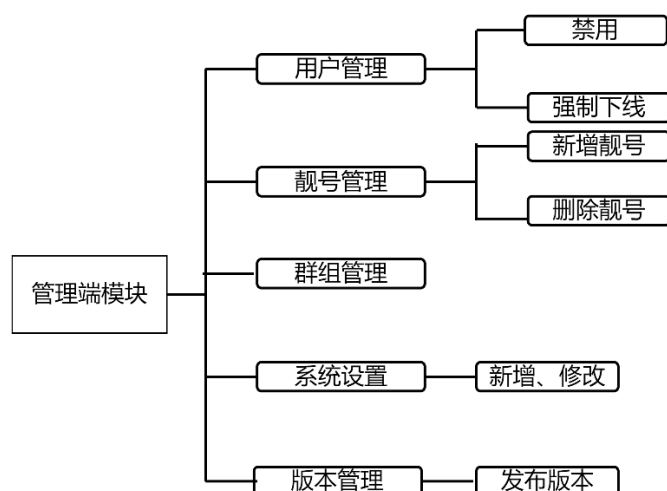


图 3.3 管理端模块功能图

(1) 系统设置

- ① 系统设置：包括对整个系统的配置管理，如语言、主题、通知设置等。
- ② 发布新版本：管理员可以进行版本更新、修复 bug 或发布新的功能版本。
- ③ 版本管理：对不同版本的系统进行管理，包括发布、维护以及灰度发布等操作。

(2) 管理后台

- ① 用户管理：管理员可以对系统内的用户进行管理，包括启用、禁用用户、强制用户下线等。
- ② 账号管理：管理员可以查看和管理所有系统用户的账号信息，包括用户权限、角色等。
- ③ 群组管理：管理员可以管理和维护群组，执行群组解散、成员管理等操作。
- ④ 靓号管理：管理员可以设置系统中特殊的靓号，包括新增和删除靓号。
- ⑤ 系统设置：管理员可以对整个系统进行配置与设置，包括修改系统配置、数据库连接、版本管理等。

(3) 消息推送与通知

- ① 消息推送：系统能够向用户推送新的消息或通知，包括用户的私聊消息、群聊消息、系统提醒等。
- ② 用户通知：除了聊天信息外，系统还支持推送用户相关的其他信息，如好友申请、群聊邀请、系统更新提醒等。

(4) 会话与会议管理

- ① 会话列表：用户可以查看所有的会话记录，包括单聊和群聊。
 - ② 删除会话：用户可以删除不再需要的聊天会话记录。
 - ③ 置顶会话：用户可以将重要的聊天会话置顶，方便快速访问。
- (5) 其他功能
- ① 搜索会话：用户可以通过搜索功能快速找到指定的聊天记录。
 - ② 设置个人信息：修改个人资料、更新头像、昵称、状态等。
 - ③ 退出系统：用户可以选择退出系统，结束当前会话。

3.3 数据库设计

3.3.1 数据库设计原则

本即时通讯系统的数据库设计严格遵循关系型数据库建模的标准范式，结合系统在高并发环境下对数据一致性、读写性能及扩展性的多重要求，构建出高可维护性、高性能的数据结构体系。作为系统的数据核心支撑，数据库有效管理用户资料、消息记录、好友关系、群组信息等关键数据，为服务端功能逻辑和客户端展示交互提供了坚实的基础。

在建模过程中，系统采用关系型数据建模方法，以实体-关系分析为基础，将用户、消息、群组、会话、系统配置等核心对象抽象为相互关联的数据表。每张表通过主键约束唯一性、外键维护数据间的逻辑关系，确保整体结构在多用户并发使用场景下依然能够保持高度的数据一致性和完整性。

数据库设计遵循以下关键原则：

(1) 规范化与反范式并举：

主要遵循第三范式行数据表划分，消除冗余数据，提升结构清晰度；同时针对离线消息、聊天记录、联系人列表等高频访问场景适度进行反范式优化，减少关联查询，提高系统响应效率。

(2) 索引优化策略：

对消息记录表、用户会话表等数据量庞大的表设置组合索引和覆盖索引，显著提升多条件查询性能；同时结合业务读写特性进行索引分布设计，避免因索引过多导致写入性能下降。

(3) 事务与一致性保障：

对于好友申请处理、消息状态更新、文件上传记录等关键业务操作启用事务控制，保证数据操作的原子性和一致性，有效避免并发写入引起的数据异常。

(4) 读写分离与缓存协同：

结合 Redis 进行高频数据，如好友状态、在线信息、群组缓存，的缓存处理，在保障数据准确性的前提下提升读取性能，同时为数据库主从架构部署与未来的读写分离做出设计预留。

(5) 安全与权限控制：

数据库层面设计细致的权限模型，防止未授权访问或敏感数据泄露，重要表设计字段级的访问隔离机制，并配合服务端进行访问控制验证。

这一数据库设计方案不仅满足即时通讯场景中对实时性与稳定性的严苛要求，同时也具备良好的扩展能力，为后续功能迭代，如聊天记录搜索、消息撤回、数据分析等，提供了强有力的结构基础。

3.3.2 系统 E-R 图

实体-关系图——E-R 图作为数据库建模的核心工具，在本即时通讯系统的数据库设计中扮演了关键角色。通过结构化的图形符号，E-R 图清晰刻画了系统中各类核心数据实体及其逻辑关联，直观展现了数据模型的组织结构与业务语义，有效支撑了系统的数据库规划与后续表结构实现。

在本系统设计中，E-R 图全面覆盖了用户、联系人、消息、群组、会话、文件等核心业务实体，细致定义了每个实体的关键属性，如用户 ID、消息类型、群组名称等，并通过实体之间的关联关系，如“发送”、“属于”、“加入”等，准确刻画业务流程下的数据交互逻辑。关系类型覆盖了一对一、一对多、多对多等多种模式，同时标注了主键、外键及唯一性等约束条件，确保数据模型的完整性和一致性。

此外，E-R 图还对弱实体，如文件、会话记录等依赖于强实体存在的数据结构，进行了明确建模，并通过适当的归类、泛化等技术处理，提高了模型的抽象能力与可扩展性。通过这种规范化的图形建模方式，系统架构师与开发团队能够在项目初期便清晰把握数据结构全貌，为数据库实现、业务逻辑编排与数据一致性控制提供了坚实的理论与技术基础。

下图所示即为本即时通讯系统核心业务模块的实体-关系图示，全面展示了系统内部的数据逻辑结构及其关联关系。

用户实体属性图如下图所示：

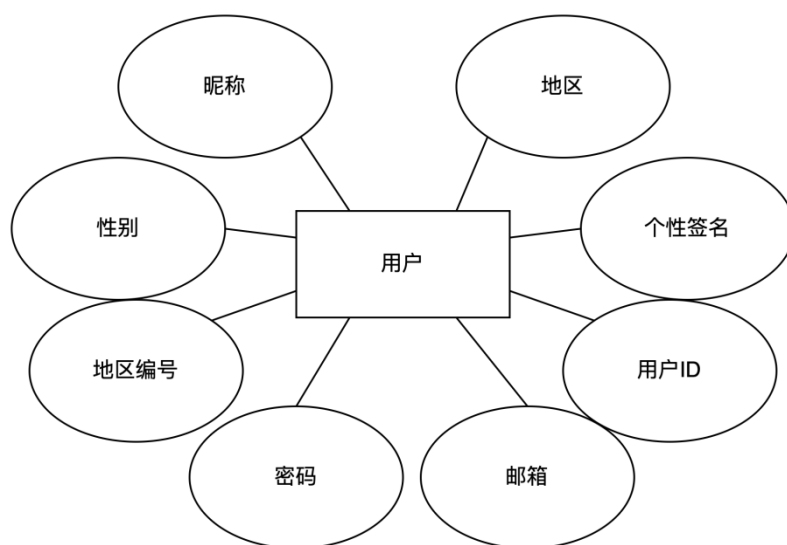


图 3.4 用户实体属性图

管理员实体属性图图如下图所示：

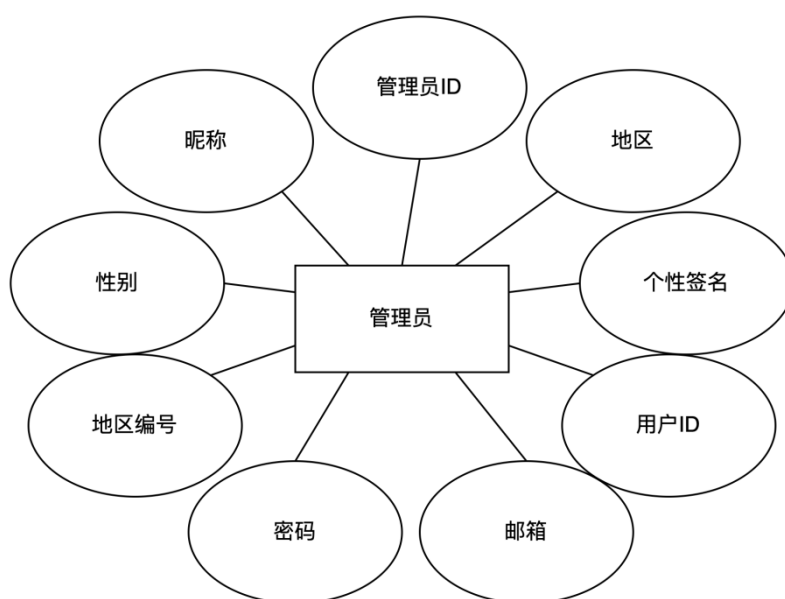


图 3.5 管理员实体属性图

联系人实体属性图如下图所示：

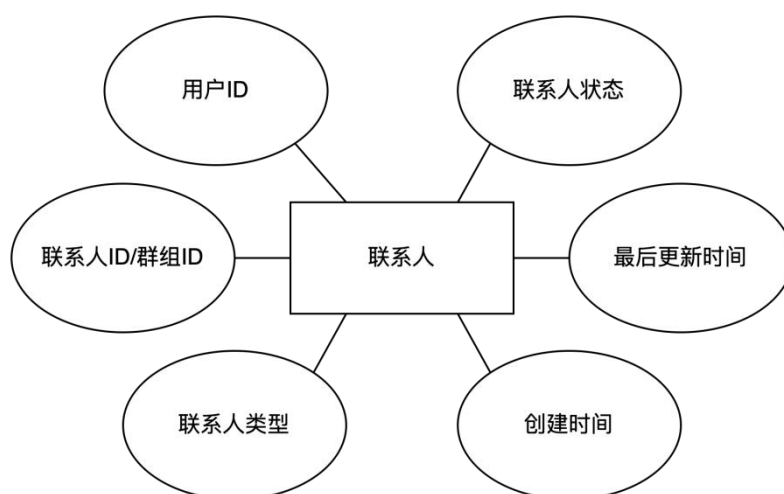


图 3.6 联系人实体属性图

群组实体属性图如下所示：

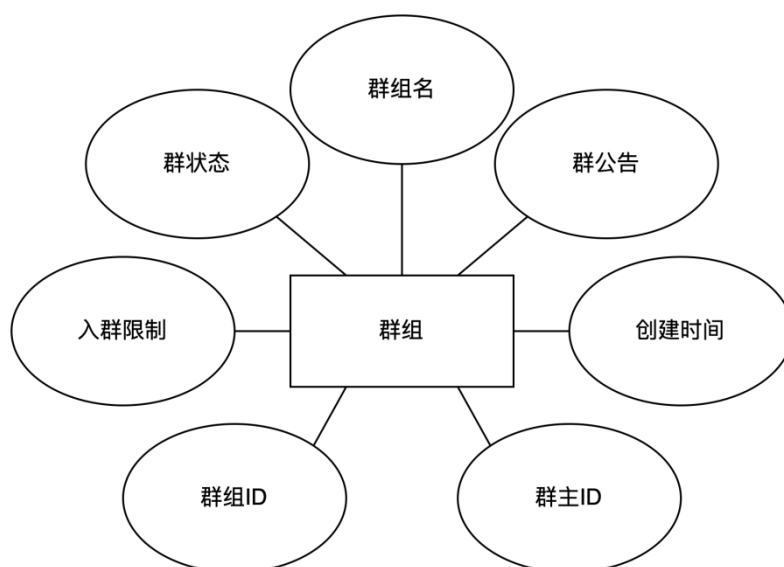


图 3.7 群组实体属性图

用户与联系人、群组 E-R 图如下图所示：

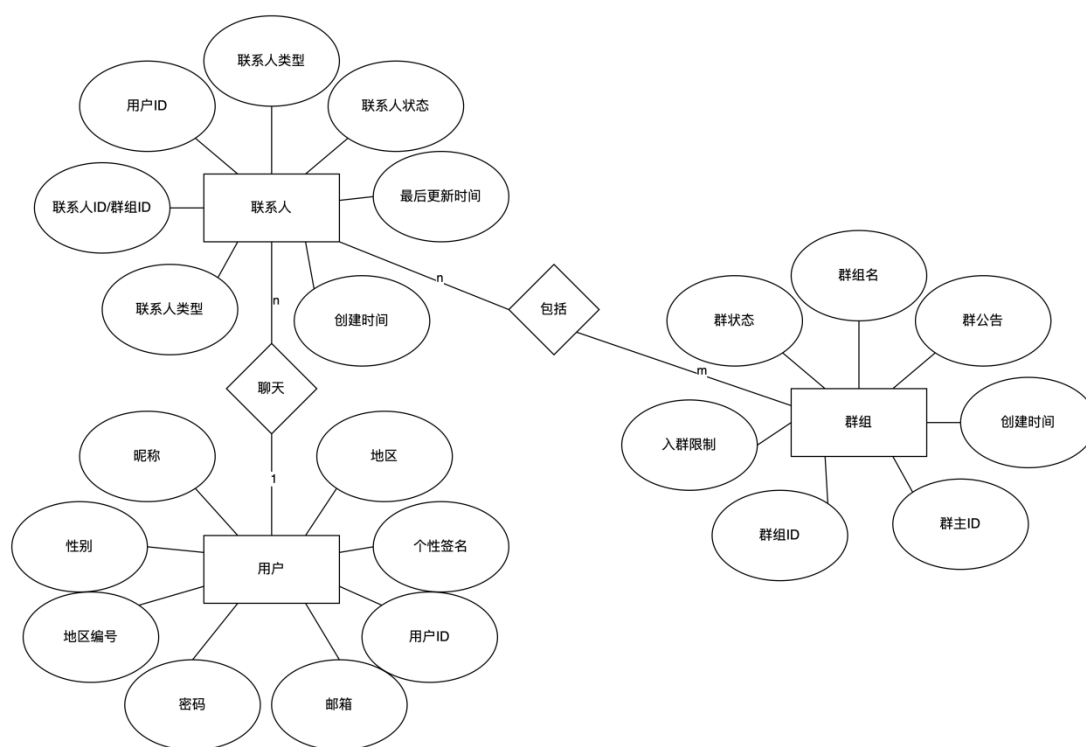


图 3.8 用户与联系人、群组 E-R 图

用户与管理员 E-R 图如下图所示：

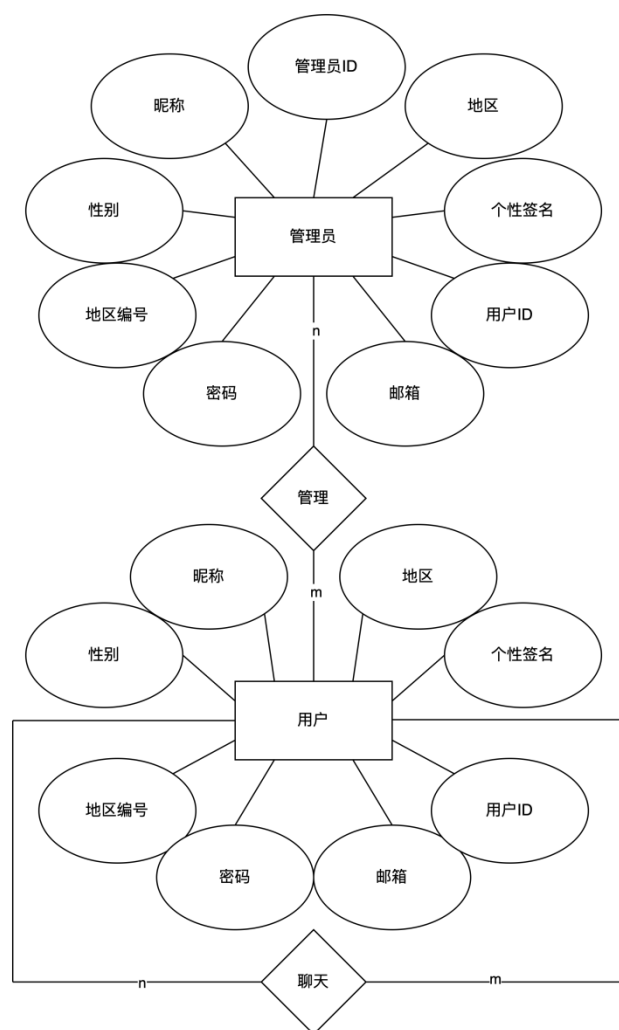


图 3.9 用户与管理员 E-R 图

3.3.3 数据库表设计

(1) App 发布信息表：用以存储更新版本号、关于更新的描述、创建时间以及发布状态、更新方式。

表 3.1 App 发布信息表

字段名称	类型	长度	是否为空	说明
id	int		否	自增 ID
version	varchar	10	是	版本号
update_desc	varchar	500	是	更新描述
create_time	datetime		是	创建时间

续表 3.1 App 发布信息表

字段名称	类型	长度	是否为空	说明
status	tinyint	1	是	0:未发布 1:灰度发布
grayscale_uid	varchar	1000	是	灰度 uid
file_type	tinyint	1	是	0:本地文件 1:外链
outer_link	varchar	200	是	外链地址

(2) 聊天信息表：用以存储系统中所有的消息记录，包括文本内容，聊天类型，发行人，发送时间，各类多媒体等基本信息。

表 3.2 聊天信息表

字段名称	类型	长度	是否为空	说明
message_id	bigint		否	消息自增 ID
session_id	varchar	32	否	会话 ID
message_type	tinyint	1	否	消息类型
message_content	varchar	500	是	消息内容
send_user_id	varchar	12	是	发送人 ID
send_user_nick_name	varchar	20	是	发送人昵称
send_time	bigint		是	发送时间
contact_id	varchar	12	否	接受联系人 ID
contact_type	tinyint	1	是	0:单聊 1:群聊
file_size	bigint		是	文件大小
file_name	varchar	200	是	文件名
file_type	tinyint	1	是	文件类型
status	tinyint	1	是	0:正在发生 1:已发送

(3) 会话消息表：用以存储最后接受的消息，最后接受消息时间毫秒，以利用时间戳实现离线接收消息的功能。

表 3.3 会话消息表

字段名称	类型	长度	是否为空	说明
session_id	varchar	32	否	会话 ID
last_message	varchar	500	是	最后接受的消息
last_receive_time	bigint		是	最后接受消息时间毫秒

(4) 会话用户表：用以存储会话用户的 ID、联系人 ID、会话 ID、联系人名称。

表 3.4 会话用户表

字段名称	类型	长度	是否为空	说明
user_id	varchar	12	否	用户 ID
contact_id	varchar	12	否	联系人 ID
session_id	varchar	32	否	会话 ID
contact_name	varchar	20	是	联系人名称

(5) 群聊信息表：用以存储群组 ID、群组名、群主 ID、创建时间、群公告、加入限制等基本信息。

表 3.5 群聊信息表

字段名称	类型	长度	是否为空	说明
group_id	varchar	12	否	群 ID
group_name	varchar	20	是	群组名
group_owner_id	varchar	12	是	群主 ID
create_time	datetime		是	创建时间
group_notice	varchar	500	是	群公告
join_type	tinyint	1	是	0:直接加入 1:管理员同意后加入
status	tinyint	1	是	1:正常 0:解散

(6) 用户连接表：用以存储用户 ID、申请人 ID、联系人 ID，便于实现多表的连接操作。

表 3.6 用户连接表

字段名称	类型	长度	是否为空	说明
id	bigint		否	自增 ID
user_id	varchar	12	否	用户 ID
apply_user_id	varchar	12	否	申请人 ID
contact_id	varchar	12	否	联系人 ID

(7) 联系人表：用以存储联系人的类型、状态、联系人 ID 等基本信息，以及最后联系时间，形成时间戳，用以判断离线后是否有新的信息更新。

表 3.7 联系人表

字段名称	类型	长度	是否为空	说明
user_id	varchar	12	否	用户 ID
contact_id	varchar	12	否	联系人 ID 或者群组 ID
contact_type	tinyint	1	是	0:好友 1:群组
create_time	datetime		是	创建时间
status	tinyint	1	是	0:非好友 1:好友 2z:已删除好友 3:被好友删除 4:已拉黑好友 5:被好友拉黑
last_update_time	timestamp		是	最后更新时间

(8) 联系人申请表：用以存储申请人用户的 ID、以及接收人用户的 ID、以及联系人的类型、最后申请时间、申请信息、申请人与接收人之间的状态。

表 3.8 联系人申请表

字段名称	类型	长度	是否为空	说明
apply_id	int		否	自增 ID
apply_user_id	varchar	12	否	申请人 ID
receive_user_id	varchar	12	否	接收人 ID

续表 3.8 联系人申请表

字段名称	类型	长度	是否为空	说明
contact_type	tinyint	1	否	0:好友 1:群组
contact_id	varchar	12	是	联系人群组 ID
last_apply_time	bigint		是	最后申请时间
status	tinyint	1	是	0:待处理 1:已同意 2:已拒绝 3:已拉黑
apply_info	varchar	100	是	申请信息

(9) 用户信息表：用以存储用户 ID、邮箱、昵称、申请条件、性别、密码、个性签名、用户状态、账号创建时间、最后登录时间、地区、地区编号、离线时间这些信息。

表 3.9 用户信息表

字段名称	类型	长度	是否为空	说明
user_id	varchar	12	否	用户 ID
email	varchar	50	是	邮箱
nick_name	varchar	20	是	昵称
join_type	tinyint	1	是	0:直接加入 1:同意后加好友
sex	tinyint	1	是	0:女 1:男
password	varchar	32	是	密码
personal_signature	varchar	50	是	个性签名
status	tinyint	1	是	状态
create_time	datetime		是	创建时间
last_login_time	datetime		是	最后登录时间
area_name	varchar	50	是	地区
area_code	varchar	50	是	地区编号
last_off_time	bigint		是	最后离开时间

(10)靓号表：用以存储管理员（靓号）的邮箱、用户 ID、使用状态这些基本信息。便于让超级管理员更好的管理管理员（靓号）。

表 3.10 靓号表

字段名称	类型	长度	字段说明	主键
id	int		否	自增 ID
email	varchar	50	否	邮箱
user_id	varchar	12	否	用户 ID
status	tinyint	1	是	0:未使用 1:已使用

第4章 系统实现

4.1 用户模块功能实现

4.1.1 登录、注册页面

下图是登录和注册页面，无论是登录还是注册用户，都得进行验证码校验，其中邮箱和密码都采用了正则表达式进行规范，如下图所示：

The figure displays two side-by-side web forms for the EasyChat application. The left form is the login page, featuring input fields for '请输入邮箱' (Please enter email), '请输入密码' (Please enter password), and '请输入验证码' (Please enter captcha) with a visual captcha '1+7=?'. It includes a green '登录' (Login) button and a link '没有账号?' (No account?). The right form is the registration page, featuring input fields for '请输入邮箱' (Please enter email), '请输入昵称' (Please enter nickname), '请输入密码' (Please enter password), '请再次输入密码' (Please re-enter password), and '请输入验证码' (Please enter captcha) with a visual captcha '5+7=?'. It includes a green '注册' (Register) button and a link '已有账号?' (Already have an account?). Both forms are titled 'EasyChat' and have a close button 'X' in the top right corner.

图 4.1 登录、注册页面示意图

4.1.2 用户功能展示

用户和管理员登录系统之后就可以查看聊天记录，查看用户列表，添加好友，创建或加入群组，删除好友，进行个人简介修改等进行详细操作，如下图所示：

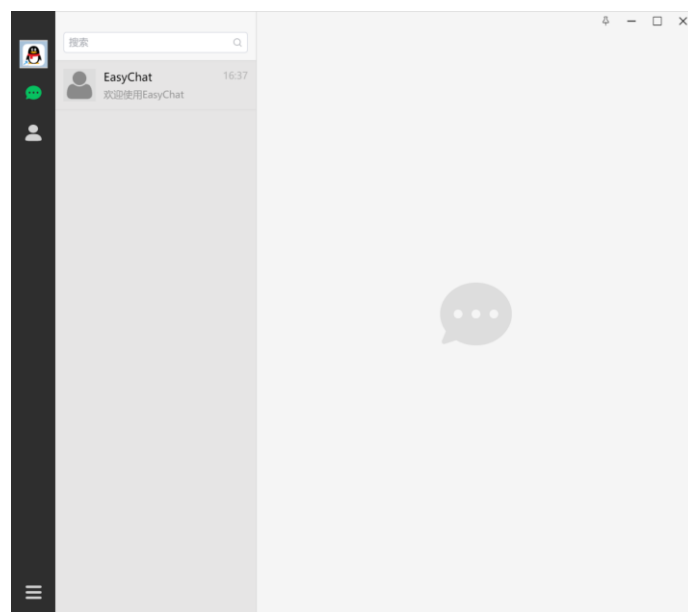


图 4.2 用户功能界面图

4.1.3 用户聊天界面

本系统的聊天管理界面采用前后端分离架构设计，前端基于 Vue3 框架构建，结合 Element Plus 实现了聊天界面布局、消息气泡、输入框、表情选择器、文件上传按钮等丰富的交互组件。前端通过 WebSocket 与后端建立长连接，实现实时消息的发送与接收，媒体类消息，如图片、视频、文件，则通过 HTTP 接口上传至服务端，并在接收端触发本地缓存与展示逻辑。

为了提升消息加载效率与用户体验，系统在客户端集成了 SQLite 本地数据库，所有接收到的聊天记录均进行本地存储，用户可快速查看历史消息，无需频繁请求服务端。对于未读消息，系统支持对会话级别的未读数进行记录与更新，在主界面中以红点提示的方式予以呈现。

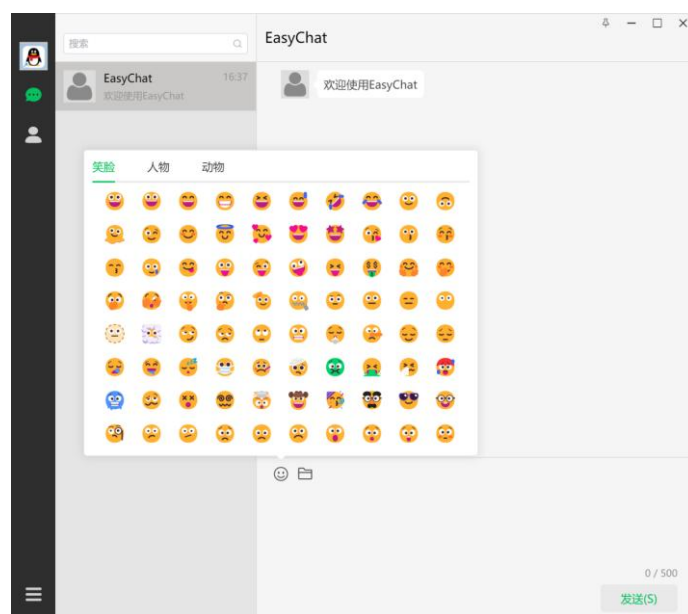


图 4.3 用户聊天功能（表情包）界面图

Electron 主进程作为服务端和渲染进程之间的中介，负责接收服务端推送的消息，统一分发到前端页面，并负责管理本地图片与视频缓存，借助本地 Express 服务实现多媒体内容的快速预览与播放。

聊天管理模块还集成了表情插入、文件发送、消息回执等核心功能，确保用户在使用过程中具备良好的实时交互体验，并能应对高并发、高频率的消息通信需求，提升了系统整体的响应速度和稳定性。通过上述设计与实现，系统在保证消息实时性的同时也有效减轻了服务端压力，实现了前端高效渲染与后端弹性处理的良性配合。

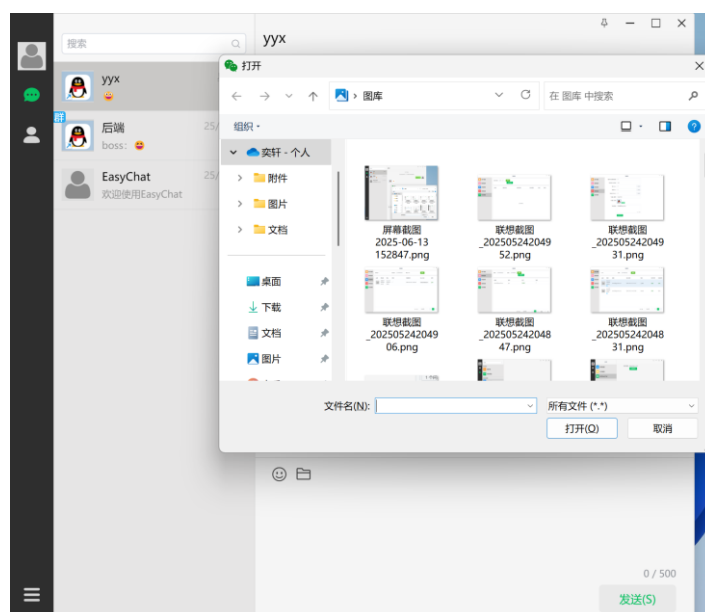


图 4.4 用户聊天功能（多媒体信息）界面图

4.1.4 用户联系人管理界面

本系统的企业管理功能采用与用户管理相似的技术架构实现，实现了结构清晰、操作便捷的联系人管理交互界面。页面涵盖好友列表展示、搜索框输入提示、好友申请处理、群组成员管理等多个功能模块，支持用户快速完成联系人相关操作。

在联系人添加方面，用户可通过关键字搜索查找好友或群组，系统支持模糊查询与结果高亮提示。申请添加好友或加入群组后，系统通过 WebSocket 实现消息实时推送，接收方可立即在界面中收到好友申请或群邀请的通知。对于收到的联系人申请，用户可直接在界面上选择“同意”、“拒绝”或“拉黑”，系统会根据用户操作实时更新联系人状态，并反馈处理结果。

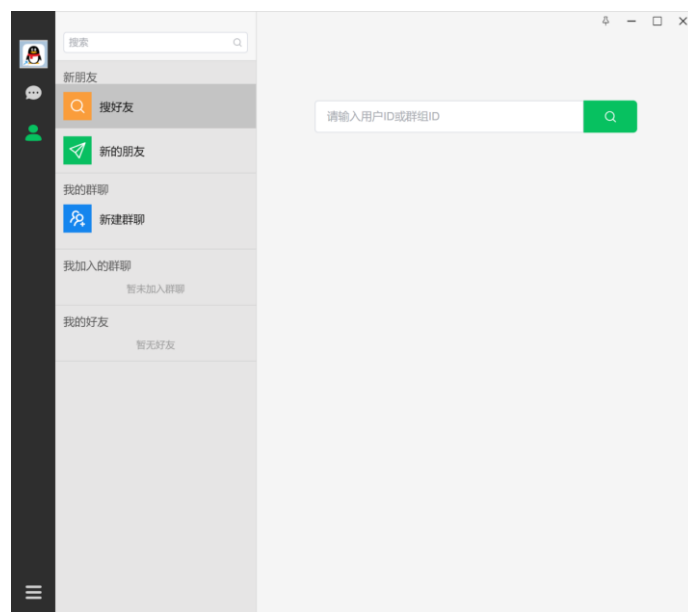


图 4.5 联系人管理（搜索好友）界面图

联系人列表采用分组展示方式，区分好友、黑名单及群组，增强用户操作的清晰度。支持对联系人执行删除、拉黑、移除群成员、解散群聊等操作，每一项操作均由前端发起 RESTful 请求交由后端处理，处理结果通过状态码和提示信息反馈至用户界面。用户在创建群聊时，可以通过复选框批量添加联系人，系统提供实时筛选与选中预览功能，提高操作效率。

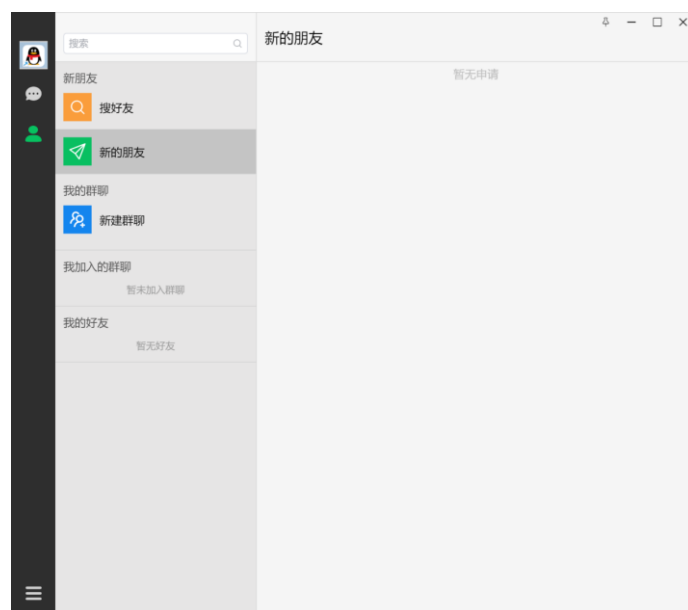


图 4.6 联系人管理（好友申请）界面图

后端采用 Spring Boot 构建，结合 Redis 缓存好友状态与群组信息，提升查询响应速度。同时，通过 Redisson 的发布-订阅机制在集群环境中同步联系人状态变更，确保多客户端数据一致性。所有联系人操作均记录在数据库中，并通过权限校验与逻辑处理保障数据安全与准确。

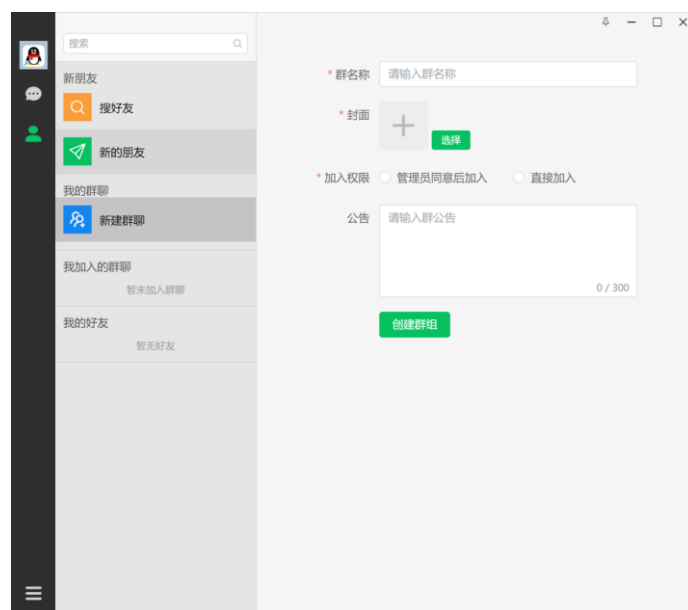


图 4.7 联系人管理（创建群组）界面图

整体而言，联系人管理界面功能完善，交互体验流畅，配合实时消息推送与高效数据缓存机制，有效满足了用户在添加、管理、组织联系人方面的需求，并支持高并发操作下的稳定运行。

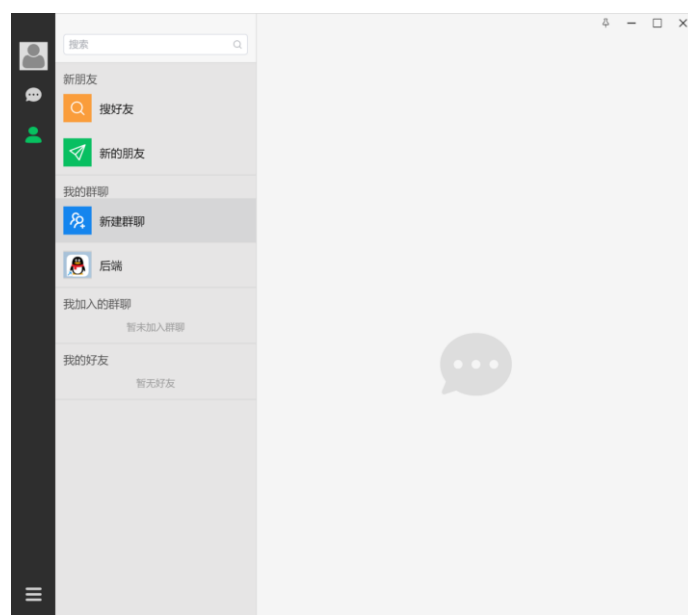


图 4.8 联系人管理界面图

4.2 管理端功能的实现

4.2.1 管理员进入界面

有些特别的用户是管理员,但只有唯一的超级管理员才能指定管理员也就是“靓号”,管理员界面进入方式如下图所示:

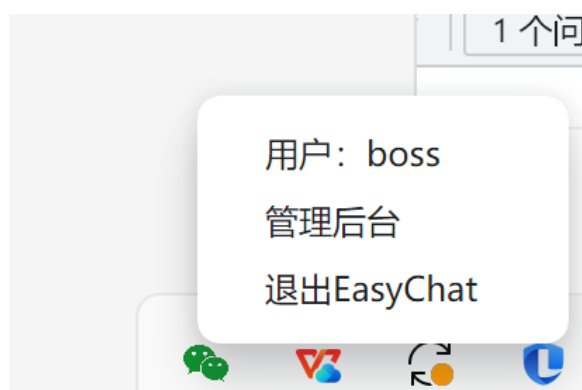


图 4.9 管理员进入界面图

4.2.2 管理员用户管理界面

管理员可以对所有用户进行管理，包括模糊查询用户信息，强制用户离线等功能，如下图所示：

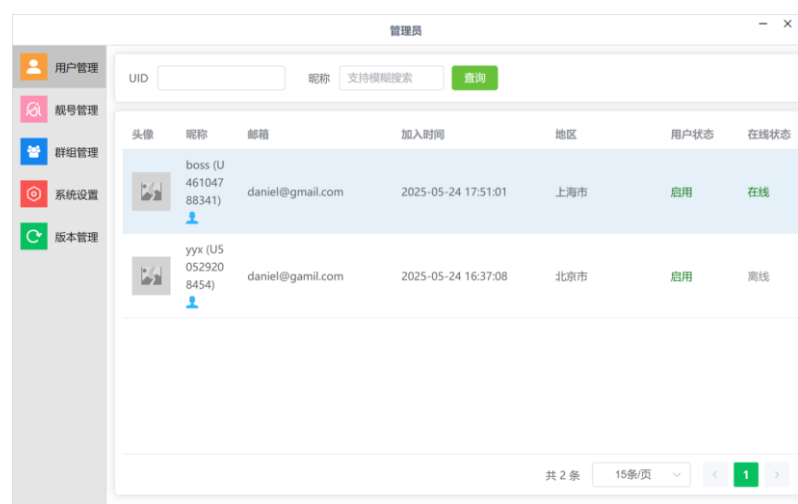


图 4.10 管理员用户管理界面图

4.2.3 超级管理员靓号管理界面

超级管理员，可以对新增，撤销靓号，也能模糊查询靓号，查看靓号对在线状态等功能，如下图所示：



图 4.11 超级管理员靓号管理界面图

4.2.4 管理员群组管理界面

管理员可以对所有群组进行管理，包括模糊查询，直接查找群主信息，所有群的基本状态的显示等。如下图所示：



图 4.12 管理员群组管理界面图

4.2.5 管理员系统设置管理界面

管理员可以设置每个人最多可创建群组数、群组最大成员数、聊天时可以发送的图片大小、视频大小以及文件大小等信息，如下图所示：

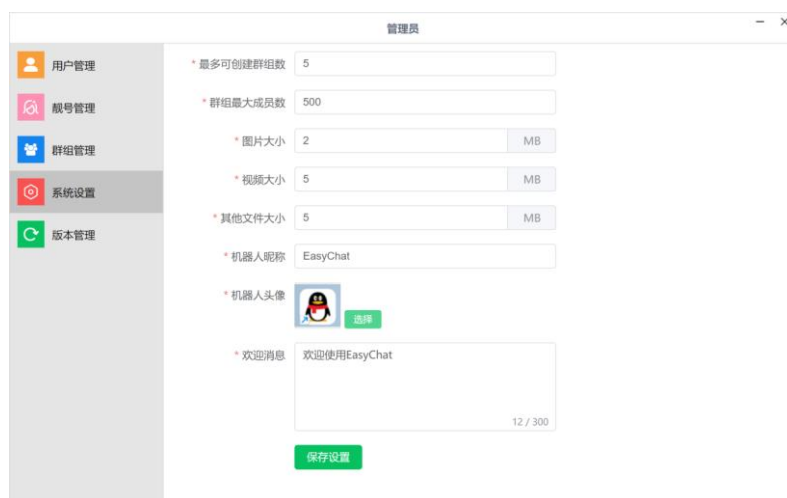


图 4.13 管理员系统设置管理界面图

4.2.6 管理员版本管理界面

管理员可通过管理后台进行新版本的新增、修改和维护。版本信息包括版本号、更新内容、适配平台、发布时间、是否强制更新等字段，系统提供了表单化的数据录入与编辑功能，并支持版本记录的分页查询与筛选操作，方便管理员快速查找历史版本或执行差异化配置。

系统支持灰度发布机制，允许管理员选择部分用户或特定客户端进行小范围试用，以验证版本的稳定性与兼容性。在灰度发布配置中，管理员可指定灰度范围，如指定账号、设备号或 IP 段，系统会自动识别符合条件的用户客户端，仅对其推送灰度版本。客户端在启动时会通过接口请求检测是否有新版本更新，并根据返回的更新策略决定是否弹出升级提示或自动下载。

后端采用 Redis 存储灰度版本策略，并结合客户端标识信息进行动态判断，确保灰度控制的灵活性与实时性。通过接口隔离与策略下发机制，即使在大规模用户并发访问的情况下，也能稳定、高效地进行版本判断与控制。同时，系统记录每次更新的覆盖范围和反馈数据，为后续版本优化提供数据支撑。如下图所示：



图 4.14 管理员版本管理界面图

第5章 系统测试

系统测试是保障软件质量的重要环节，它能够帮助开发团队及时发现并解决系统中的潜在问题，从而确保系统在上线前达到预期的性能和质量标准。系统测试不仅是一项技术性工作，更是保障用户在实际使用过程中获得良好体验的重要手段。通过全面的测试过程，可以有效验证系统的稳定性与各项功能，确保产品发布后能够正常运行，满足用户的使用需求并实现设计目标。

5.1 系统测试内容

5.1.1 用户登录注册测试设计

登录时，当账号密码未填写、填写错误或验证码输入错误时用户无法登录。注册时，必填项未填写时系统会提醒必填项未填写，进而导致注册不成功，只有必填项全部填写且按照正确格式填写后，输入正确的验证码，才能保证注册成功。用户登录注册测试设计如表 5.1 所示。

表 5.1 用户登录注册测试设计表

测试点	测试方案	理想结论	真实结论	结论分析
用户登录	账号: yyx 密码: yyx	点击登录后出现提示“账号或密码错误”	点击登录后出现“账号或密码错误”	账号或密码出错不可登录
用户登录	只输入账号	点击登录出现提示“密码只能是数字、字母、特殊字符 8~18 位”	点击登录出现提示“密码只能是数字、字母、特殊字符 8~18 位”	密码不可为空
用户登录	只输入密码	点击登录出现提示“请输入正确的邮箱”	点击登录出现提示“请输入正确的邮箱”	账号不可为空
用户登录	账号密码都为空	点击登录出现提示“请输入正确的邮箱”	点击登录出现提示“请输入正确的邮箱”	账号密码都不能为空
用户登录	账号密码输入正确，验证码输入错误	点击登录出现提示“图片验证码不正确”	点击登录出现提示“图片验证码不正确”	账号密码输入正确，但验证码输入错误

续表 5.1 用户登录注册测试设计表

测试点	测试方案	理想结论	真实结论	结论分析
用户登录	账号密码输入正确，验证码输入正确	点击登陆按钮后，成功登录	点击登陆按钮后，成功登录	账号密码为必填项需要输入正确，验证码也必须输入正确
用户注册	只输入邮箱	点击注册，提示“密码只能是数字、字母、特殊符号 8~18 位”	点击注册，提示“密码只能是数字、字母、特殊符号 8~18 位”	用户昵称、密码为必填项

5.1.2 用户联系人管理测试设计

用户可以对联系人进行管理包括搜索好友，添加新朋友，新建群聊，联系人列表展示，搜索联系人等一系列操作。当添加新朋友，或者群聊时，如果已被对方拉黑，是否会有提示，同时也测试了精准查询功能，用户联系人管理测试如表 5.2 所示。

表 5.2 用户联系人管理测试表

测试点	测试方案	理想结论	真实结论	结论分析
用户联系人管理	点击联系人列表按钮	显示所有的已加入的群聊和已加好友	显示所有的已加入的群聊和已加好友	联系人列表展示通过测试
用户注册	只输入邮箱	点击注册，提示“密码只能是数字、字母、特殊符号 8~18 位”	点击注册，提示“密码只能是数字、字母、特殊符号 8~18 位”	用户昵称、密码为必填项
用户联系人管理	点击新建群聊，直接点击创建群组	会提示“请输入群名称”，“请选择头像”，“请选择加入权限”	会提示“请输入群名称”，“请选择头像”，“请选择加入权限”	创建群聊时，群名称、封面、加入权限是必填项
用户联系人管理	点击新建群聊，输入群名称，选择封面，选择加入权限后点击创建群组	会提示群组创建成功，联系人列表展示出来新建的群组	会提示群组创建成功，联系人列表展示出来新建的群组	新建群聊功能通过测试

续表 5.2 用户联系人管理测试表

测试点	测试方案	理想结论	真实结论	结论分析
用户联系人管理	点击搜好友，输入用户 ID: yyx，此用户已将测试用户拉黑	会显示对应用户信息，旁边显示已被拉黑	会显示对应用户信息，旁边显示已被拉黑	被拉黑用户无法添加对方为好友
用户联系人管理	点击搜好友，输入用户 ID: daniel	会显示对应用户信息，旁边有添加按钮	会显示对应用户信息，旁边有添加按钮	搜索好友功能通过测试
用户联系人管理	点击新朋友，点击忽略	会显示所有申请信息，点击忽略后该新朋友申请信息消失	会显示所有申请信息，点击忽略后该新朋友申请信息消失	添加朋友功能中的忽略朋友功能通过测试
用户联系人管理	点击新朋友，点击拉黑	会显示所有申请信息，点击拉黑，下次将不会收到改用户的申请信息	会显示所有申请信息，点击拉黑，下次将不会收到改用户的申请信息	添加朋友功能中的拉黑朋友功能通过测试
用户联系人管理	点击新朋友，点击添加	会显示所有申请信息，点击添加，该用户将出现在你的联系人列表中	会显示所有申请信息，点击添加，该用户将出现在你的联系人列表中	添加朋友功能中的添加朋友功能通过测试

5.1.3 用户设置管理测试设计

用户可以进行账号设置，文件管理，关于 EasyChat 版本管理的功能，用户可以通过账号管理功能进行密码的修改，以及一些个人简介的修改，还能推出登录。文件管理则是可以自由改变本地存储的位置。版本管理，可以检测当前更新版本。用户设置管理测试设计表如表 5.3 所示。

表 5.3 用户设置管理测试设计表

测试点	测试方案	理想结论	真实结论	结论分析
用户设置管理	点击账号设置，点击退出登录	从用户聊天界面退出，回到登陆界面	从用户聊天界面退出，回到登陆界面	退出登录功能通过测试

续表 5.3 用户设置管理测试设计表

测试点	测试方案	理想结论	真实结论	结论分析
用户设置管理	点击账号设置， 点击修改密码，输入修改后密码：“1”，随后点击修改密码按钮	提示“密码只能是数字、字母、特殊字符 8~18 位”，“请再次输入密码”	提示“密码只能是数字、字母、特殊字符 8~18 位”，“请再次输入密码”	密码，确认密码必不可少。而且密码必须受一定的规范约束
用户设置管理	点击账号设置， 点击修改密码，输入修改后密码：“a1234567!”， 确认密码：“a123456!” 随后点击修改密码按钮	提示“两次输入的密码不一致”	提示“两次输入的密码不一致”	密码和确认密码必须一致
用户设置管理	点击账号设置， 点击修改密码，输入修改后密码：“a1234567!”， 确认密码：“a1234567!” 随后点击修改密码按钮	出现窗口提示“修改密码后将退出登录，需要新登录，确认要修改吗？”点击确认，回归到用户登录界面	出现窗口提示“修改密码后将退出登录，需要新登录，确认要修改吗？”点击确认，回归到用户登录界面	修改密码功能通过测试
用户设置管理	点击修改个人信息，昵称改为空， 点击保存个人信息	提示“请输入昵称”	提示“请输入昵称”	昵称为必填项，不能为空
用户设置管理	点击修改个人信息，昵称改为“yyx”， 点击保存个人信息	提示“保存成功”	提示“保存成功”	修改个人信息功能通过测试
用户设置管理	点击文件管理， 点击打开文件夹	出现本地缓存所在的文件夹的位置	出现本地缓存所在的文件夹的位置	文件管理寻找本地缓存功能通过测试
用户设置管理	点击文件管理， 点击更改，点击选择文件夹	弹出文件窗口，自行选择电脑中任意文件夹，文件管理目录发生变化	弹出文件窗口，自行选择电脑中任意文件夹，文件管理目录发生变化	文件管理更改本地缓存路径功能通过测试

5.1.4 用户聊天管理测试设计

用户可以在联系人列表中，任意选择群组或者用户进行聊天，包括发送文本消息，视频、图片等多媒体消息，文件等。即使用户离线，也能接收到相关信息，等到用户重新上线时会有消息提醒。用户聊天管理测试设计表如表 5.4 所示。

表 5.4 用户聊天管理测试设计表

测试点	测试方案	理想结论	真实结论	结论分析
用户聊天管理	选择群组“后端”，点击表情包，发送“笑脸”	群组其他成员同步看见“笑脸”	群组其他成员同步看见“笑脸”	表情包发送功能通过测试
用户聊天管理	选择群组“后端”，点击文件，选择一个视频文件进行发送	群组其他成员同步看见视频，能进行下载播放	群组其他成员同步看见视频，能进行下载播放	多媒体信息发送功能通过测试
用户聊天管理	选择群组“后端”，在输入文本框中输入文字“欢迎大家”，点击发送	群组其他成员能同步看见文字“欢迎大家”	群组其他成员能同步看见文字“欢迎大家”	文字发送功能通过测试
用户聊天管理	选择联系人“yyx”，输入文本信息“你好”，点击表情包“笑脸”点击发送，随后点击文件，选择一张图片进行发送	用户 yyx，能看见信息“你好‘笑脸’”和图片信息	用户 yyx，能看见信息“你好‘笑脸’”和图片信息	单聊功能通过测试
用户聊天管理	选择联系人“yyx”，发送任意信息，联系人“yyx”不在线	用户 yyx 重新上线后，会收到消息提示，并能得知是谁发送的信息	用户 yyx 重新上线后，会收到消息提示，并能得知是谁发送的信息	消息提醒功能通过测试

5.1.5 管理员用户管理测试设计

管理员可以查看所有用户对基本信息、加入时间以及用户状态。而且管理员可以用 UID 查询，或者昵称对模糊查询，查找相关用户。管理员用户管理测试设计表如表 5.5 所示。

表 5.5 管理员用户管理测试设计表

测试点	测试方案	理想结论	真实结论	结论分析
管理员用户管理	点击用户管理，搜索“U46104788341”	显示出用户“boss”的基本信息，加入时间，用户状态	显示出用户“boss”的基本信息，加入时间，用户状态	UID 精准查询功能通过测试
管理员用户管理	点击用户管理，搜索“y”	显示出所有昵称含“y”的用户	显示出所有昵称含“y”的用户	昵称模糊查询功能通过测试

5.1.6 超级管理员靓号管理测试设计

管理员可以查看所有管理员（靓号）的邮箱、昵称、状态等信息，并且支持靓号和邮箱的模糊查询。超级管理员（仅有一个）能新增管理员（靓号）或者删除管理员（靓号）。超级管理员靓号管理测试设计表如表 5.6 所示。

表 5.6 超级管理员靓号管理测试设计表

测试点	测试方案	理想结论	真实结论	结论分析
超级管理员靓号管理	点击靓号管理，搜索“y”	显示出所有靓号含“y”的管理员的信息	显示出所有靓号含“y”的管理员的信息	靓号模糊查询功能通过测试
超级管理员靓号管理	点击靓号管理，搜索“daniel”	显示出所有邮箱含“daniel”的管理员的信息	显示出所有邮箱含“daniel”的管理员的信息	邮箱模糊查询功能通过测试
超级管理员靓号管理	点击靓号管理，点击删除靓号	被操作的靓号失去了管理员权限	被操作的靓号失去了管理员权限	删除靓号功能通过测试
超级管理员靓号管理	点击靓号管理，新增靓号，输入靓号邮箱	被操作的用户获得管理员权限	被操作的用户获得管理员权限	靓号添加功能通过测试

5.1.7 管理员群组管理测试设计

管理员可以通过群组 ID, 群主 UID 的精准查询和群名称的模糊查询, 查找到系统内任何符合条件的群组的群名称, 群人数, 群状态, 创建时间等所有的基本信息。管理员群组管理测试设计表如表 5.7 所示。

表 5.7 管理员群组管理测试设计表

测试点	测试方案	理想结论	真实结论	结论分析
管理员群组管理	点击群组管理, 搜索 “G70249555539”	显示群组 ID 为 “G70249555539” 的群名称、群人数、群主信息、创建时间、群状态等 基本信息	显示群组 ID 为 “G70249555539” 的群名称、群人数、群主信息、创建时间、群状态等 基本信息	群组 ID 精准查询功能通过测试
管理员群组管理	点击群组管理, 搜索 “后”	显示出所有群名称含“后”的群的基本信息, 包括: 完整的群名称、群人数、群主信息、创建时间、群状态等 基本信息	显示出所有群名称含“后”的群的基本信息, 包括: 完整的群名称、群人数、群主信息、创建时间、群状态等 基本信息	群名称模糊查询功能通过测试
管理员群组管理	点击群组管理, 搜索 “U46104788341”	显示所有群主 UID 为 “U46104788341” 的群组的基本信息, 包括: 群名称、群人数、群主信息、创建时间、群状态等基本信息	显示所有群主 UID 为 “U46104788341” 的群组的基本信息, 包括: 群名称、群人数、群主信息、创建时间、群状态等基本信息	群主 UID 精准查询功能通过测试

5.2 系统测试结论

本设计的开发完成后, 对其进行了全面的系统测试, 涵盖了账号管理、联系人管理、聊天功能、个人信息管理、管理后台操作等多个模块, 旨在验证系统在功能完整性、运行稳定性、安全性和用户体验等方面的表现。

首先, 在用户登录与注册模块, 系统能够准确验证用户所输入的账号密码信息。测试过程中, 对于空输入、错误账号、错误密码等常见输入场景, 系统均能够及时给出明

确提示，并阻止无效请求的继续处理，保障了用户身份验证的安全性和交互的友好性。同时，注册流程中对账号格式、密码强度的校验逻辑也被逐一验证，确保了用户信息录入的规范性和系统的稳定运行。

其次，在联系人管理模块中，包括创建群组、添加移除群成员、申请好友、删除联系人、拉黑处理等功能，通过黑盒测试和边界条件测试，系统均能正确响应用户操作，保持联系人数据的一致性和准确性。例如，当用户尝试重复添加同一联系人或重复加入群聊时，系统能准确识别并给予限制提示，提升了系统的可控性和使用体验。

在聊天功能方面，系统支持发送文本、表情、媒体文件等多种类型的消息，通过 Netty 实现的消息推送功能经多轮测试后表现稳定，能够保证消息的实时传达。对于媒体消息的发送与接收流程，我们重点测试了大文件的上传与分发机制，验证了消息状态的正确同步以及上传过程的容错能力。此外，离线消息推送功能通过模拟不同时间段的用户下线与重连场景，确认服务端可以正确记录最后离线时间并基于此推送离线期间的消息内容，保障了消息完整性与用户体验的连续性。

在客户端部分，系统通过 Electron 主进程与服务端建立 WebSocket 长连接，测试中验证了消息实时传输的稳定性以及媒体文件的本地缓存效果。通过 SQLite 实现的本地消息缓存也在多轮历史消息加载测试中表现出良好的读取速度与一致性，用户在不同场景下都能快速查看聊天记录，明显降低了对服务端的压力。

在后台管理系统测试方面，包括强制下线、用户启用禁用、靓号管理、版本检测与发布等功能都进行了功能性和权限控制测试，验证了不同角色用户访问权限的有效限制以及后台操作对前端系统的影响反馈，系统反应准确，功能表现一致。

最后，在系统的整体稳定性测试过程中，我们对高并发登录、连续消息发送、多客户端同时操作等复杂业务场景进行了模拟测试，系统在高负载下仍保持良好的响应能力和资源调度，未出现内存泄漏、服务崩溃等严重问题。同时，异常场景处理逻辑完备，如断网重连、服务器异常响应等情况下均可恢复连接并保证消息数据不丢失。

综上所述，本设计在功能性、稳定性、安全性及用户体验方面均通过了系统测试的全面验证，满足当前产品发布与运行的要求。虽然测试过程中仍发现个别界面细节、交互逻辑可进一步优化，但整体系统性能可靠、操作流畅，具备良好的用户适配能力和可维护性，已具备正式上线运行的条件。

第6章 技术经济分析

6.1 复杂工程问题研究

本系统满足复杂工程问题特性的第一条、第四条和第七条的内容，具体说明如下：

(1) 第一条特性要求：必须运用深入的工程原理，经过分析才可能得到解决。

本项目在架构设计阶段即体现出对工程原理的深入运用。为满足高并发、实时性要求，我们选择了 Netty 作为底层通信框架，配合 WebSocket 协议实现客户端与服务端的双向长连接通信，有效支持消息的实时推送。同时，服务端采用基于分布式架构设计，利用 Redis 缓存存储用户状态、好友关系和系统设置，借助 Redisson 的 RTopic 实现集群下的消息广播，确保系统在多节点部署时依然能保持一致性和稳定性。

此外，离线消息推送功能则通过记录用户最后的离线时间戳，再在用户重新上线时拉取对应时间段内的消息，实现精准、按需推送，避免了传统轮询方式带来的系统开销。媒体文件的消息流转更是结合了异步处理、文件缓存和状态同步机制，保障用户体验的同时减轻服务器压力。

(2) 第四条特性要求：不是仅靠我们经常使用的方法就可以完全解决的。

传统的 HTTP 短连接通信机制难以满足即时通讯场景对低延迟和高稳定性的需求。因此，我们采用了 WebSocket 长连接技术，并基于 Netty 进行自定义协议设计，以满足复杂消息结构、心跳机制、断线重连等关键功能。

同时，在多设备登录、群聊成员管理、好友申请等复杂状态管理问题上，仅依靠简单的数据库操作无法高效完成。因此我们引入了 Redis 作为中间缓存层，配合发布-订阅机制和状态同步逻辑，实现系统内外状态一致性的维护。版本更新与灰度发布功能也借助多维度规则控制与客户端反馈机制，突破了传统一次性更新策略的局限，实现了对特定用户分阶段推送更新版本。

(3) 第七条特性要求：具有较高的综合性，包含多个相互关联的子问题。

本系统包含账号管理、联系人管理、聊天管理、系统版本控制、后台管理等多个模块，功能繁多且相互依赖。例如，用户登录系统后需同步获取好友列表、聊天会话和未读消息，这涉及账号系统、好友关系缓存、消息存储与推送系统的联动操作。

在联系人模块中，添加好友、加入群聊、处理申请等操作依赖于身份验证、权限判断和用户状态同步，要求系统具备良好的模块通信机制和一致性处理能力。聊天模块与文件服务紧密耦合，消息发送前需确认媒体文件是否上传成功，再同步消息状态到客户

端。此外，管理后台的用户强制下线、群组解散与靓号管理等功能也需要与前台系统实时通信，反映出系统间的高度耦合性与协作性。

综上所述，该即时通讯系统在技术选型、架构设计、功能实现等方面均涉及工程原理的深度应用，采用了多种先进技术手段突破传统方法的限制，并在多模块高耦合条件下实现了系统的稳定、高效运行，符合复杂工程问题的多个典型特性。

6.2 技术可行性分析

本设计是一款基于即时通讯的跨平台桌面应用系统，具备用户登录注册、好友管理、群组管理、消息通信、文件传输、后台管理等多项功能，涉及前后端协作、实时通信、文件处理、本地缓存及系统版本控制等复杂业务场景。为了实现系统的高性能、高并发与良好用户体验，项目从架构设计、技术选型到实现机制均进行了充分的技术可行性论证，具体分析如下：

(1) 后端技术可行性

后端基于 Spring Boot 框架开发，具备快速开发、自动配置、开箱即用等优点，适合构建 RESTful API 服务。同时结合 Redis 实现状态缓存与心跳管理，Redisson 提供分布式发布-订阅机制，在集群环境下保障消息推送的一致性与高可用性。采用 JWT Token 实现用户身份验证，有效提升系统安全性。通过 Netty 构建长连接通信通道，支持高并发、低延迟的消息推送。使用 MySQL 完成用户数据、聊天记录与好友关系的持久化存储，保证系统数据的一致性与可靠性。

(2) 客户端技术可行性

客户端采用 Electron 框架实现跨平台桌面应用，其主进程负责与服务端通信、维护本地服务，并通过与渲染进程的数据交互完成消息与媒体处理。“Vue3 + Element Plus”用于构建渲染进程的 UI 界面，提升界面响应效率与交互体验。Pinia 实现统一状态管理，axios 用于 HTTP 接口请求，前端逻辑清晰、易于维护。通过 WebSocket 与服务端建立实时连接，接收推送消息。多媒体消息缓存至本地，通过 Node Express 启动本地服务实现图片、视频的预览播放，结合 SQLite 存储本地聊天记录和消息状态，大幅降低服务端压力并提高查询效率。

(3) 实时通信与离线消息可行性

系统采用 Netty 保持客户端与服务端的长连接，确保消息实时送达。同时，服务端记录每位用户的最后离线时间，通过时间戳筛选离线期间的消息，在用户重新上线后及时推送，确保消息不丢失。该机制已在多个高并发场景中得到验证，具备良好的扩展性与稳定性。

(4) 系统版本控制与灰度发布可行性

系统引入版本控制功能，结合后台的版本检测模块支持版本的新增、修改与灰度发布策略，确保在不影响整体用户体验的情况下逐步上线新功能或修复补丁。通过服务端记录客户端当前版本信息，可实现定向推送更新，保障系统升级的可控性与安全性。

6.3 经济可行性分析

本设计主要服务于团队协作与消息交流，适用于中小型组织或私有部署的办公场景。由于其应用范围相对明确，系统所承载的并发请求和数据量并不属于大规模范畴，因此对硬件资源的依赖较低。普通配置的计算机或服务器即可满足部署需求，进一步降低了使用门槛与资源成本。

在项目开发阶段，系统全面采用了开源技术栈，包括前端的“Vue3 + Element Plus”、Electron 框架、后端的 Spring Boot、Netty、Redis、MySQL，以及本地数据库 SQLite 等。这些开源工具不仅功能完备、社区活跃，且无需额外支付授权费用，有效控制了开发成本。开发团队可以将更多精力投入在系统功能的完善与用户体验的优化上，而非许可证或商用授权问题。

此外，系统在运行过程中设计了高效的资源使用机制，例如客户端使用本地缓存减少服务端压力、Redis 缓存优化读写性能、Netty 长连接支持高效消息推送等。这些优化手段确保了系统在实际使用中对服务器资源的消耗最小化，从而减少了运维所需的硬件投入。

运维方面，系统具备良好的可扩展性与稳定性，支持分布式部署与灰度更新机制。日常维护仅需对 Redis 缓存、数据库存储以及服务器健康状态进行监控，人员投入相对有限。结合日志管理与自动化脚本处理，大大减少了人工干预频率和维护成本。

综上所述，本设计基于开源方案开发，部署资源要求低，运维成本可控，在保障高效通信与功能完整的前提下，具备显著的经济可行性，适合中小规模组织作为内部通信工具长期稳定使用。

第 7 章 设计总结

历时几个月，顺利完成了本次毕业设计的全部工作内容，包括毕业设计说明书的撰写以及即时通讯系统的完整开发与实现。本系统面向桌面端用户，基于 Electron 技术构建，具备注册登录、联系人管理、消息通信、媒体文件传输与管理后台等多项功能，体现了系统设计的综合性和工程实现的复杂性。

在设计开发初期，从需求出发，对即时通讯系统的核心功能进行了全面梳理，明确了系统应包含的模块：包括用户账号体系、联系人管理机制、个人信息设置、实时聊天与文件传输功能、以及后台管理系统等。通过对多个同类产品的分析，构建了功能完整、逻辑清晰的系统功能框架。

在技术选型方面，选择 Spring Boot 作为后端核心框架，结合 Redis 进行状态缓存与数据同步，使用 Netty 实现高性能的消息推送机制，充分保障了系统的并发性能与稳定性。前端方面，采用 Vue3 与 Element Plus 实现用户界面设计，通过 Pinia 进行状态管理；同时利用 Electron 构建跨平台的桌面应用，并在主进程中引入 Node.js 和本地 Express 服务，为多媒体文件的本地缓存与预览提供了坚实支撑。在本地数据管理上，使用 SQLite 作为客户端数据库，提升了消息读取效率与离线访问能力。

本设计采用 token 实现用户鉴权，结合 Redis 实现好友关系、系统设置及客户端心跳数据的缓存管理；同时基于 Redisson 的 RTopic 模块，实现分布式集群环境下的消息发布与订阅。Netty 长链接保障了消息通信的实时性，离线消息机制通过记录用户最后离线时间并基于时间戳拉取数据，在用户重新登录时快速恢复通信状态，极大提升了用户体验。媒体类消息通过先行上传再分发状态的方式，有效解决了大型文件传输过程中的一致性和及时性问题。

客户端部分的实现也具有一定挑战性。Electron 主进程作为桥梁连接服务端与渲染进程，既要管理文件缓存、消息缓存，又需承担本地服务启动任务，为视频播放和图片预览提供支持。同时在 UI 设计中采用响应式交互模式，确保系统界面在不同设备分辨率下保持良好的视觉体验和操作流畅性。

在设计开发的后期，重点进行了系统测试与性能调优工作。通过模拟高并发环境测试系统的消息推送与接收能力，验证了 Netty 框架下的高效性与稳定性。同时对客户端的消息加载与渲染逻辑进行优化，保障在大量历史记录情况下依旧能快速响应用户操作。

综上所述，本毕业设计中的即时通讯系统开发过程，覆盖了从系统需求分析、架构设计、技术实现到部署测试的完整开发流程。项目过程中我深入理解了分布式消息系统、WebSocket 实时通信、本地缓存机制与前后端协作开发的技术细节。

展望未来，随着即时通讯需求的不断增长与场景多样化，系统可进一步拓展移动端适配、语音视频通话、AI 智能助手等功能模块，同时通过引入微服务架构与容器化部署技术，提升系统的扩展能力与运维效率。通过持续的技术演进与产品优化，系统有望成为具备企业级能力的跨平台即时通讯解决方案。

参 考 文 献

- [1] 凌敏,王骥.基于 Netty 和 MongoDB 的车联网 Web 系统[J].测绘与空间地理信息,2021,44(10):55-58+62.
- [2] Hornsteiner M , Kreussel M , Steindl C ,et al.Real-Time Text-to-Cypher Query Generation with Large Language Models for Graph Databases[J].Future Internet, 2024, 16.DOI:10.3390/fi16120438.
- [3] 张方兴.高性能 Java 架构:核心原理与案例实战[M].电子工业出版社,2021.
- [4] Ruh J, Steiner W, Fohler G. Clock synchronization in virtualized distributed real-time systems using IEEE 802.1 AS and ACRN[J]. IEEE Access, 2021, 9: 126075-126094.
- [5] Deshpande K , Jain A ,Abhinav,et al.Empowering Real-Time Communication: A Seamless Chatting System Using Websocket[C]//International Conference On Innovative Computing And Communication.Springer, Singapore, 2025.DOI:10.1007/978-981-97-4152-6_29.
- [6] Jnr. B A , Sylva W , Watat J K ,et al.A Framework for Standardization of Distributed Ledger Technologies for Interoperable Data Integration and Alignment in Sustainable Smart Cities[J].Journal of the Knowledge Economy, 2024, 15(3):12053-12096.DOI:10.1007/s13132-023-01554-9.
- [7] 杨开振,刘家成.Java EE 互联网轻量级框架整合开发 SSM+Redis+Spring 微服务(全 2 册) 网络技术[M].电子工业出版社,2021.
- [8] 罗娇燕,左黎明,陈艺琳,等.基于 SM9 的 JWT 强身份认证方案[J].计算机技术与发展, 2024, 34(3):110-117.
- [9] 凌伟博.基于 Raft 算法的分布式消息队列研究与实现[D].北京市:北京林业大学,2022.
- [10] 黄世中.一种基于 RESTful 的轻量级身份认证方法[J].电子设计工程, 2020(9):18-21.

致 谢

行文至此，落笔之时，我的大学生活即将圆满结束。这篇论文的完成不仅标志着我学术旅程的重要节点，更是我人生新阶段的起点。此刻，我满怀感恩之情，谨向在我成长道路上给予帮助和支持的人们表达最诚挚的谢意。

首先，我要衷心感谢我的导师——赵明老师。从课题的确定到研究的深入，从论文的结构设计到最终的文字润色，赵老师始终给予我悉心的指导与不懈的鼓励。赵老师渊博的学识、严谨的治学态度和和蔼可亲的为人风范，深深感染并影响着我。在我遇到迷茫和困难的时候，是赵老师一如既往地给予我指引和支持，让我能够坚持下来、走到今天。赵老师不仅是我学业上的引路人，更是我人生道路上的一盏明灯。

我还要深深感谢我的家人。感谢他们在我成长过程中默默的付出和无私的爱，是他们给予我坚实的后盾和无限的力量。在我遇到挫折时，是他们的鼓励让我重拾信心；在我取得进步时，是他们的陪伴让我倍感温暖。他们的理解和支持是我不断努力、不懈追求的源泉。

同时，我也要感谢学校提供的良好学习环境和丰富的资源保障。在图书馆里查阅资料的日子，在教学楼里调试程序的时光，都是我成长道路上珍贵的回忆。感谢所有任课教师和工作人员，是你们的辛勤付出与默默奉献，才有了我们今天的收获与成就。

回首四年的大学时光，有过迷茫、有过坚持，也有过感动与收获。这段经历不仅丰富了我的知识体系，更让我学会了独立思考、勇敢面对挑战。论文的写作过程虽然充满艰辛，但也让我更加坚定了前行的方向。

未来的路还很长，我将继续怀揣感恩之心，铭记这段时光赋予我的一切力量与希望，勇敢走向新的旅程。无论前方风雨如何，我相信，凭借在这里积累的知识、能力和信念，我定能迎难而上，追逐梦想，砥砺前行。

附录一 外文原文

附录二 中文译文